

Coffre - A Distributed System for In & Out Labs

*A Bachelor Thesis Report
by*

Vamsi Bedapudi and Bhavan Turaka
(150050068 and 150050091)

Supervisor:
Prof. Uday Khedker



Department of Computer Science and Engineering
Indian Institute of Technology Bombay
Mumbai 400076 (India)

14 May 2019

Acknowledgements

We thank Prof. Uday Khedker for providing us with a wonderful opportunity to work on this project.

We thank Ashok Kumar C, Ph.D candidate under Prof. Bernard Menezes and Prof. G. Siva Kumar, at IIT Bombay for providing insights about use and scope of SGX in our project. We thank Dr. Vishwas Patil, Senior Research Scientist at IIT Bombay for giving us valuable knowledge regarding Signed Kernels and othe Security Measures.

We thank Prof. Purushottam Kulkarni, Prof. Kameswari Chebrolu, Prof. Bhaskaran Raman for giving us valuable insights during the project development

Vamsi Bedapudi and Bhavan Turaka

IIT Bombay

14 May 2019

Abstract

Coffre is a BTP project started under the guidance of Prof. Uday Khedker with the aim of providing the students an isolated, distributed environment where they can write their code and make submissions without copying from one another or from the internet. Coffre is a French word which means VAULT.

The student should not be able to copy text or files from our isolated environment to any outside system or vice-versa. The student should be able to use this isolated environment in a distributed manner, being able to access it on any system he/she wishes to. He/She should be able to safely store the work done in this isolated environment in an online manner. The student should be able to make submissions from the isolated environment, whether it is an assignment or a project.

Contents

I	Introduction & Motivation	5
1	Motivation	6
2	Introduction	6
2.1	Goals	6
3	Virtual Machines vs Docker	7
3.1	What does Virtual Machine (VM) mean?	7
3.2	Oracle VM VirtualBox	7
3.3	Docker	7
3.4	Why we chose Virtual Machine over Docker	7
4	Architecture	7
II	Client Tools & Security	9
5	User Authentication	10
5.1	Definition	10
5.2	Known Methods of Authentication	10
5.2.1	Password Based Authentication	10
5.2.2	Shared Key Authentication	10
5.2.3	Public and Private Key Authentication	10
5.2.4	Comparison	10
6	User Levels & File Permissions	11
6.1	User Levels in Linux	11
6.1.1	Root User	11
6.1.2	Regular User	11
6.1.3	Service Account	11
6.2	Groups in Linux	11
6.3	File Ownership in Linux	11
6.3.1	User	11
6.3.2	Group	11
6.3.3	Other	11
6.4	File Permissions in Linux	12
6.4.1	Read	12
6.4.2	Write	12
6.4.3	Execute	12
7	Tools for Secure Communication	12
7.1	SSH	12
7.2	GIT	12
8	Need for SSH	13
8.1	Attack Scenario	13
8.2	Solution for the Attack	13
8.3	Git	13
9	User Authentication in Coffre	13
9.1	Comparison of Authentication Methods	13
9.2	Attack Scenario	13
9.3	Solution for the Attack	14
9.3.1	How SSH uses public and private keys	14
9.3.2	Main Idea	14
9.3.3	Implementation	14

10	Disabling Interaction Between Host and Guest	14
10.1	VirtualBox Guest Additions	14
10.2	Preventing data transfer between host OS and guest OS	15
11	Disabling External Media Mounting	15
11.1	Attack Scenario	15
11.2	Mounting External Media in Linux	15
11.3	Blacklisting Kernel Modules in Linux	15
11.4	Idea and Implementation of the Solution	15
12	Disabling Network Access	16
12.1	Attack Scenario	16
12.2	Solution for the Attack	16
12.2.1	iptables	16
12.2.2	UFW	16
12.2.3	Main Idea	16
12.2.4	Implementation	16
13	Preventing Man In The Middle Attack	17
13.1	Attack Scenario	17
13.1.1	IP spoof Attack	17
13.1.2	Man-in-the-middle Attack	17
13.1.3	SSH defences	17
13.1.4	Problems with SSH defences	17
13.1.5	Main Attack	17
13.2	Main Idea and Implementation of the Security Measures	18
14	Enabling Grub Security	18
14.1	Attack Scenario	18
14.1.1	Booting into recovery mode	18
14.1.2	Using GRUB to invoke bash	19
14.2	Idea and Execution of the Solution	19
15	Full Disk Encryption	19
15.1	Mounting a filesystem on Live CD	19
15.2	Full Disk Encryption (FDE)	19
15.3	Usability problem with FDE	19
15.4	Solution	20
16	Mandos	20
16.1	How much extent Mandos fits for us	20
16.2	How Mandos doesn't fit for us	20
16.3	Possible Attack	20
III	Server Tools & Security	21
17	Securing SSH connection on server	22
17.1	AUTHORIZED_KEYS file in ssh	22
17.2	Disabling shell access, -X option to the user in SSH connection	22
17.2.1	Port forwarding	22
17.2.2	X11 forwarding	22
17.2.3	Agent forwarding	22
17.2.4	Pseudo-tty access	23
18	Git Server	23
18.1	Why local git server	23
18.2	What is Gitosis	23
18.3	What is Git Daemon	24
18.4	Handling public repositories	24

19 Various Client applications for handling submissions	25
19.1 Professor client application	25
19.2 Moodle Student client application	25
19.3 Moodle TA client application	25
20 Submission server	25
20.1 What is SCP	25
20.2 SCP for file submissions	26
20.3 Changing timestamp attack and its prevention	26
20.4 Changing filename attack and its prevention	26
20.5 Server overloading	26
20.6 Handling partial submission	27
20.7 Handling simultaneous submissions by the same group	27
20.8 Handling simultaneous submissions by multiple groups	27
20.9 Interaction of server script with SQL database	27
20.10SQL injection attack and its prevention	27
IV Client Setup & Update	28
21 Setup Script	29
21.1 Why do we require the setup	29
21.2 Initial approach	29
21.3 Optimal approach	29
21.4 Securing the setup	29
21.5 Another possible attack	29
21.6 What if 'setup' script is abruptly stopped in the middle	29
22 Updates	30
22.1 Why updates	30
22.2 Sudoers file	30
22.3 How we are using sudoers file to run update script as a root user	30
22.4 Version Control	30
22.5 Securing the updates	30
V Conclusion	31
23 Preventive measures if Mandos passphrase is leaked	32
23.1 User identifies private key passphrase from SSH binary	32
23.2 User cannot identify the private key passphrase from SSH binary	32
23.2.1 Check if the SSH binary is running in a VirtualBox environment	32
23.2.2 Check if the User has gained ROOT access	32
23.2.3 Check if the User has introduced any new files into the system	32
24 Software Guard Extensions (SGX)	33
24.1 Introduction to SGX	33
24.2 Current State of SGX	33
24.3 How SGX can be used in our system	33
25 Future work	33
25.1 Usability	33
25.2 Student Montioring	33
25.3 Creating preventive measures if encryption passphrase is leaked	33
25.4 SGX	34
26 Conclusion	34
VI Coffre User Manual & FAQs	35

Part I
Introduction & Motivation

1 Motivation

The goal of the project is to provide a healthy environment for the students, that prevents them from copying from one another by sharing files. This project more or less tries to solve the same problem MOSS solves, which is not allowing students to copy assignments from one another or from the internet. As the famous saying prevention is always better than cure goes, this project is a prevention method while MOSS is a cure for the above problem.

There already exists some applications which provides an isolated environment, for writing code, but in a centralized environment. One such software is Prutor developed under Prof. Amey Karkare for teaching introductory programming. Prutor cannot be used for doing medium to heavy projects. It provides only a text editor and doesn't support whole capabilities of an Operating system like multiple text editors and applications. Prutor works in a centralized manner, where connection to the central server is always necessary for a prutor client to work.

In courses like Implementation of programming Languages Lab, there is a requirement that students have to work from home on out-labs. The students prefer to work offline and later submit the work they did. Keeping this in mind, we chose to make a distributed environment rather than a centralized environment (i.e keeping everything on server) for solving the problem of copying. Though it is easier to emphasize security on a centralized environment rather than a distributed one, it's tradeoff between security implementation (distributed environment needs to handle more security issues than centralized one) and flexibility.

2 Introduction

Our project, Coffre aims to provide students an isolated environment where they can write their code in an isolated system. Each student/group gets their own copy of their Virtual Disk Image (VDI) file with non-root access. The OS present in the VDI file is Ubuntu 16.04. They can load it on any Virtual Machine and start using it.

2.1 Goals

The goals, we want to meet in this project are:

- The students/groups should be able to work in a distributed manner, having multiple copies of their Virtual Machine, running on multiple systems. For example, a student/group can have a Virtual Machine on their lab machine and another in their personal computer.
- The student/group shouldn't be able to interact with the host OS (They shouldn't be able to copy text or files from host OS to our Virtual Machine and vice versa).
- The student/group shouldn't be able to have internet access from within the Virtual Machine.
- The student/group shouldn't be able to mount any external media such as USB sticks, CD-DVD's to transfer files in and out of the Virtual Machine.
- The student/group shouldn't be able to tamper with any other student's submissions or files.
- Students/Groups shouldn't be able to transfer files among them, however hard they may try.
- The student/group should have access to a private git repository, which he/she can use to sync files between multiple copies of their Virtual Machine.
- The student/group should be able to make submissions for the labs in a course like Implementation of Programming Languages

3 Virtual Machines vs Docker

3.1 What does Virtual Machine (VM) mean?

A virtual machine (VM) is a software program or operating system, that not only exhibits the behavior of a separate computer but is also capable of performing tasks such as running applications and programs, like a separate computer. A virtual machine, usually known as a guest is created within another computing environment referred as a "host". A virtual machine is also known as a "guest". A virtual machine runs on a host with the help of the hypervisors like Oracle VM VirtualBox

3.2 Oracle VM VirtualBox

Oracle VM VirtualBox is a free and open-source hosted hypervisor for x86 virtualization, developed by Oracle Corporation. Created by Innotek GmbH, it was acquired by Sun Microsystems in 2008, which was, in turn, acquired by Oracle in 2010. VirtualBox may be installed on Windows, macOS, Linux, Solaris and OpenSolaris.

3.3 Docker

Docker is a tool designed to make it easier to create, deploy, and run applications by using containers. Containers allow a developer to package up an application with all of the parts it needs, such as libraries and other dependencies, and ship it all out as one package. By doing so, thanks to the container, the developer can be rest assured that the application will run on any other Linux machine regardless of any customized settings, that machine might have that could differ from the machine used for writing and testing the code.

3.4 Why we chose Virtual Machine over Docker

Equivalent entities	
Docker	VirtualBox
Docker container	Virtual Machine
Docker image	Virtual Disk Image (VDI)

Whether its the Docker Container or the Virtual Machine, we need a mediator to run the image on the host computer. For Docker Container, the mediator is the Docker itself while for the Virtual Machine, the mediator is any hypervisor like Virtualbox. We don't have any control over the mediator. It runs on the user machine and they have full control over it. So whatever security we have to incorporate, it has to be in the image itself.

When we had a discussion with Prof. Amay Karkare, we found out that its easy for the user to get root permission for the docker image. Once the user gets the root permission, all the security can break. With Docker, anyone can log into the Docker container using the root user (ID = 0) instead of the provided default root user when they use the -u option.

E.g. `docker exec -u 0 -it mycontainer bash`. In a Virtual Machine, we have more scope and flexibility to put all the restrictions to prevent the user from getting the root access. So we chose Virtual Machine over Docker.

4 Architecture

The architecture of our project is as shown in Figure 1. It contains a Student Client (which is a Virtual Machine), a Professor client (which is a Django website) and a TA client (which is a Terminal application). On the server, there are git server, submission server, Django web server and a mandos server. The access control in git server is controlled by a tool called Gitosis.

The data flow is as follows:

1. When the student Virtual Machine boots, it contacts the mandos server to fetch the passphrase which is used to decrypt its Virtual Hard Disk. Mandos server is explained in section 16.
2. Professor adds labs and their deadline using Django web client that interacts with the Django server and the data corresponding to it is stored in SQL tables. Django client and webserver are explained in section 19.

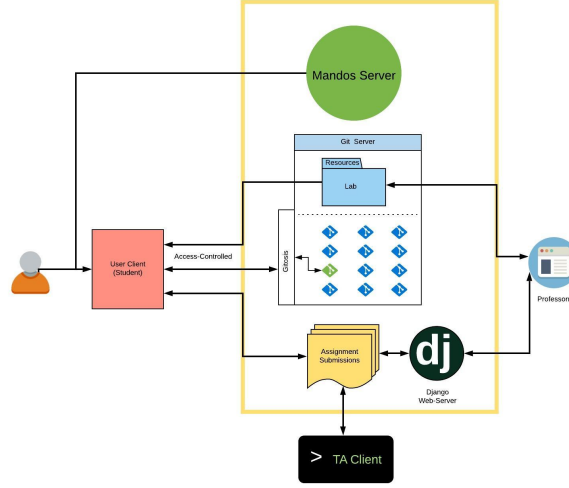


Figure 1: Coffre Architecture

3. Students can sync their work into their online private repository by connecting to the git server. Git server is explained in section 18
4. Students can make submissions to the labs added by the professor by connecting to the submission server which fetches lab data like list of available labs, their submission deadline etc from SQL tables. Submission server is explained in 20
5. TAs can view and download the student submissions through TA client by connecting to the submission server. TA client is explained in section 19

The design of the architecture is an output of extensive discussion between the team members of the project. Generally implementation of the security measures is trivial once a security hole and its solution is known. But finding out security hole and getting to the best solution is done only when worked in a collaborative manner filled with discussions. Though there is no clear boundary in the work done by us for the project, we can say the major part of the Client Security is taken care by Vamsi Bedapudi and the major part to the Server Security is taken care by Bhavan Turaka. The challenging security issues which don't have solutions as of now have been researched by both of us by meeting various people with excellent skills in their respective fields.

Part II
Client Tools & Security

5 User Authentication

5.1 Definition

Authentication means verifying the identity of users who want to access some data, resources or applications of a system. Generally, users authenticate and identify themselves by providing a token/-some data, which only they are supposed to have. To call an authentication method secure, it should in no way or form allow a malicious user to spoof himself as another user without providing the user's authentication token/data.

5.2 Known Methods of Authentication

5.2.1 Password Based Authentication

General known authentication mechanisms, say logging into a website, use a password which is set by the user, supposed to be only known to him/her and thus secure to authenticate users. Each user is assigned a user id, which can be his/her name or a Gmail id in case of Gmail. This user id and password combination is then checked against the server's/system's database which contains a hash of the actual password with the user id, and the user is authenticated only when the password matches with the corresponding entry in the database.

There is another well-known method of authentication, known as Key authentication. Key authentication involves either a shared key or a public and private key pair.

5.2.2 Shared Key Authentication

In shared key systems, the user and authenticator contain the same shared key, and some cryptographic functions are used by the authenticator to verify that the user has the same key as the authenticator. In this method to distinguish N users that authenticator must have N different keys, as otherwise the authenticator can easily mix up user identities or a malicious user can spoof himself as another user with the same shared key.

5.2.3 Public and Private Key Authentication

There is another variant of the key authentication called asymmetric key authentication. In this authentication both the user and authenticator have a pair of public and private keys. The public key can be known to everyone, but the private key is known only to the user. These keys are generated securely using one-way cryptographic functions. The public key is used not only in authenticating the user but also in identifying the user. When compared to the password-based authentication we can say that public key acts as a user id, while the private key acts as the corresponding password. For a malicious user to be able to spoof himself as another victim user, the malicious user must know the victim's private key.

5.2.4 Comparison

When we take a comparison of all the above three discussed methods of authentication, shared key setting is impractical in an N users scenario, due to the hassle and difficulty of maintaining N different keys by the authenticator. Compared to passwords, private and public key authentication is a bit more secure as the user tends to keep simple and easily guessable passwords. Compared to passwords a private key is 2048/4096 bits in length and is randomly generated, thus not guessable, and so more secure.

6 User Levels & File Permissions

6.1 User Levels in Linux

Linux is a multiuser operating system. That is more than one user can use Linux at a time. There are three types of user in Linux:

1. Root User
2. Regular User
3. Service Account

6.1.1 Root User

This is the main user account in Linux system. A root user has the highest privilege in the system. The root user can do any administrative work and can access any service. It can't be deleted. But if required, it can be disabled.

6.1.2 Regular User

This is the normal user account. There is no limit on the number of regular user accounts. These have moderate privileges. These can perform only the tasks for which it is allowed and can access only those files and services for which they are authorized.

6.1.3 Service Account

Service accounts are created by programs during installation if they are installed. These accounts are used by services to run processes and execute functions.

6.2 Groups in Linux

In Linux, every user is associated with at least one or more groups. Authorizations and permissions can be set on a per group basis also.

6.3 File Ownership in Linux

Every file and directory on a Linux system is assigned three types of owners.

1. User
2. Group
3. Other

6.3.1 User

A user is the owner of the file. By default, the person who created a file becomes its owner.

6.3.2 Group

A group can contain multiple users. All users belonging to a group will have the same access permissions to the file or directory.

6.3.3 Other

Any other user who can access the file. This person has neither created the file nor belongs to a user or a group who could own the file

6.4 File Permissions in Linux

Every file and directory in a Linux system has three permissions defined for the three owners discussed above.

1. Read
2. Write
3. Execute

6.4.1 Read

This permission give a user the authority to open and read a file. Read permission on a directory gives the user the ability to lists its content.

6.4.2 Write

The write permission gives a user the authority to modify the contents of a file. The write permission on a directory gives a user the authority to add, remove and rename files stored in the directory.

6.4.3 Execute

The Execute permission on a file gives a user the authority to run the file, but neither modify the file or read the contents of the file. The execute permission on a directory gives a user the authority to chdir to that directory.

7 Tools for Secure Communication

7.1 SSH

Secure Shell (SSH) is a network protocol. It uses cryptography to enable, operating network services safely and securely over an unsecure network. Unsecured network refers to the trustability of the network, i.e., if we can safely send data over the network with the guarantee that no-one other than the communicating parties can access or modify the data traveling over the network without raising an error in the communication. Generally, all networks are insecure. More importantly, communication on the internet is unsecured, if it happens without using any secure cryptography.

The SSH protocol is typically used to execute commands securely on a remote machine. But it can do a lot more, it also enables secure file transfer and secure copy protocols, which can be used to transfer files across the internet securely. Any network service can be made secure using SSH protocol.

SSH supports password-based and public and private key based authentication.

7.2 GIT

Git is a simple and popular distributed version control system. It can be used to coordinate work among multiple people working in a team. It can be used to track changes across a series of files, store and retrieve the older copy of files. Git, as a distributed version control system, not only helps in tracking and maintaining versions of files in a single system, but also helps in keeping track of the changes to files and different versions across multiple independent systems.

Git provides these version control features via the help of repositories. Git is to be initialized in a directory, which Git calls a repository. All the files in this directory/repository can be tracked by git for version management purposes. New files can be added to this repository or old files can be deleted. Git can be used from multiple systems, to keep track of the same repository. Git enables this sync of a repository between multiple systems, by keeping a remote repository on the server and syncing all the repositories local to the systems with the remote repository to maintain sync between local repositories. Thus by adding a file to a local repository in one system, will make it appear in other local repositories of other systems using the remote repository.

Git manages access and permissions to remote repositories via user authentication methods such as password-based authentication and key based authentication.

8 Need for SSH

When a user submits his/her assignments to the server in our system, the user and we hope that his/her submission goes unchanged to the server and also that no other person should accidentally or intentionally read or modify what the user has submitted to the server. A similar security guarantee is needed for the files in the user's git repository. That is no one should be able to read the files in the user's git repository or modify the files in his/her git repository.

8.1 Attack Scenario

One way a malicious entity can access the user's submissions is by listening to network traffic sent by the user, and from it decoding the files submitted by the user. Similarly, he/she may try to change the user's submission by modifying the network traffic. He/She may also try to sniff or listen to the network traffic from the git program and try to access or modify user's git repositories.

8.2 Solution for the Attack

Only a secure network can protect itself against these kind of attacks/accesses and give us the security guarantees the user and we have hoped for. This is where SSH comes to the rescue, as it secures any communication between a client and a server over any unsecured network.

8.3 Git

Git also can be made to use SSH for its communication with the server, when it is trying to sync its local repository with the remote repository on the server. We can also configure Git to use the same authentication used by SSH in authenticating the client, to authenticate a Git user on the same client to the server.

Thus all of our security requirements have been met by the use of SSH.

9 User Authentication in Coffre

If we take a look back at the above-discussed methods of authentication in User Authentication Section, we can see that all the authentication methods involve two common elements, one some way for the user to identify himself, two some way for the user to prove who he/she says he/she is. All the above methods are great and can generally be used for authentication. We should note that in all the methods discussed above, the user knows or can easily know his password or private key. The above methods are all secure, in the perspective of a malicious user trying to pose as a victim user.

9.1 Comparison of Authentication Methods

Since using shared keys means the server has to maintain N shared keys one each for the N users, we decided to abandon this method of authentication. Since password-based authentication involves user typing in the password every time, he wants to perform an SSH action which are, running git commands and submitting to the server; it becomes cumbersome to use password-based authentication. So, we decided to go with a private and public key based authentication.

9.2 Attack Scenario

Two users user M1 and user M2, are using our system. Now using our authentication mechanisms, we can stop the user M1 from posing as M2, if M2 doesn't wish it. But if M2 willingly shares his (M2's) public and private key with M1, then M1 can easily authenticate himself as M2 and access M2's files and resources.

A primary requirement as mentioned in the Introduction section of our system is that, a user must not be able to share files with the other users. Following the above attack, if M2 gives his private key to M1, M1 can authenticate to our system as M2. M1 can then download and look at submissions of M2 or can access and download files from M2's git repository. Then M1 can log out as M2, re-authenticate himself as M1 and submit the files downloaded from M2 as his creations to our server.

This attack on our system, where user's M1 and M2 collaborate and exchange files has been made possible because M1 and M2 know their private key which the server uses for user authentication.

9.3 Solution for the Attack

So, to stop this attack, the only way is to authenticate user M1, but without the user M1 being able to read the private key used to authenticate him/her.

9.3.1 How SSH uses public and private keys

SSH implementations readily support public and private key based authentication. The public and private keys which are to be used by the SSH program, have to be placed in `/home/<user>/.ssh` directory, from where the program takes the keys to be used for authentication.

Since an SSH program has to be able to read the public and private keys to authenticate the user, and the ssh program has the same level permissions as that of the user running it, the public and private keys can be read by the user.

And since a user who can access his public and private key gives us a security risk, we should somehow stop either the user M2 sharing the private key of M2 with M1 or make the shared private key of M2 useless to M1.

9.3.2 Main Idea

Public and Private keys that are used by the SSH program can be encrypted with a passphrase, but this passphrase has to be supplied every time to the SSH program when it wants to use the public and private keys for authentication.

To stop the user from knowing the passphrase, which the keys are encrypted with, we can hardcode the passphrase into the SSH binary. Since, a user may be able to analyze the SSH binary for the passphrase, using Linux file permissions we can set the SSH binary to be only made executable and no more readable, editable to the user. This will prevent the user from being able to get his/her passphrase by analyzing the SSH binary program.

We can specify a separate passphrase for each user and thus make sharing the keys useless without the passphrases. So, now even if M2 shares his/her private and public key pair with M1, M1 will not be able to use them mimicking M1, as M1's and M2's public and private key passphrases differ.

9.3.3 Implementation

OpenSSH is an open source, suite of secure networking utilities, which are based on the Secure Shell (SSH) protocol. Since this is an open-source program suite; its source code is easily available. The code can be easily modified and recompiled to create a custom version of OpenSSH program.

OpenSSH has a `sshconnect2.c` file in which it requests the user for a passphrase. This request mechanism was bypassed for a user, and the user's passphrase was hardcoded as the passphrase of the encrypted private key of the user. Then the resulting program was compiled, and the entire set of tools were generated. The compiled binaries are kept in `/usr/local/bin`, as the location is by default placed in path and thus the user can call upon the SSH program without much care for its location.

This passphrase to be hardcoded for each user is separate, and thus the resulting binaries are different. Each binary will be placed in the user's system with only execute permissions given to the user, thus securing passphrase for the public and private keys.

Note: SSH programs require that the permissions on the private and public keys used by them to be readable and accessible only to the user. Thus the user is given both read and write access to his public and private key files. Even if they are edited by the user, the system is secure as the passphrase is unknown to the user.

10 Disabling Interaction Between Host and Guest

10.1 VirtualBox Guest Additions

The Guest Additions are some extra bits of software that you install in the operating system you're virtualizing. That operating system is also called the guest OS. This is where the name Guest Additions comes from - they're additions to the guest OS. Installing the Guest Additions enables various extra features that are already built into VirtualBox. These include automatic resizing, seamless mode, the bidirectional clipboard, drag, and drop, shared folders support etc.

10.2 Preventing data transfer between host OS and guest OS

We don't want extra features provided by the Guest Additions in our Virtual Machine as we don't want the user to transfer data between host OS and guest OS through clipboard or through file sharing. By default, the user can't copy-paste using the clipboard and can't drag and drop files from host OS to guest OS or vice-versa without Guest Additions. So by preventing the user, from installing the Guest additions on the guest OS, we can prevent copy-paste using clipboard and file sharing between host OS and guest OS. We did it by blacklisting kernel drivers that are needed for mounting the Guest Additions ISO file. The blacklisting of the kernel drivers can't be reversed by the user as he/she doesn't have root access to the guest OS.

11 Disabling External Media Mounting

11.1 Attack Scenario

Now consider the scenario with two users M1 and M2. M1 and M2 cannot share their keys with each other. This is the security guarantee provided by the system until now.

Imagine the case; User M1 attaches a USB stick on to M2's system, M2 mounts it. M2 then transfers his submission files to the USB stick. After the files have finished copying now, the user M1 takes out the USB stick, inserts it into his own system, mounts the USB stick and copies the contents from the USB stick to his own system. Now M1 has access to M2's submissions files.

11.2 Mounting External Media in Linux

This attack is possible, in spite of all the security guarantee's above as we did not enforce any rules on mounting/using of any external media on to the system.

In Linux, each device such as a USB/hard disk/CD-DVD, connected to the system is associated with a device file. These device files are stored in `/dev`. For Example `/dev/cdrom` may be the device file of a cd inserted into the system. These device files are just RAW pointers to the devices inserted. Reading and writing directly from these device files is almost impossible as one has to implement the entire logic of an ext4 file system to do so, i.e., one has to implement the logic for finding inodes, accessing the block structure and then updating the inode.

The mount command facilitates mounting operations of device files for any external media. When external media such as USB/hard disk/CD-DVD are inserted into the system, mount program initializes a whole set of kernel operations, which read from the external media's device file in `/dev`, identify the filesystem used by the device, initialize the filesystem for the device and then allow the user to access it in the location `/media/<device-file-name>`. Thus enabling normal file system access to the device.

The mount command by default loads all the devices in `/media/<device-file>` directories. The `/mnt` directory present in Linux systems acts as a temporary mount point in this process of mounting a device file into the `/media` directory.

To use the mount command to load a device file into any other directory than `/media`, say into the `/Downloads` directory requires root access, which is enforced by the mount command itself. Thus without a root user access, a normal user can only mount files in `/media` directory.

11.3 Blacklisting Kernel Modules in Linux

Sometimes, for various performance or security reasons, it is necessary to prevent the system from using a certain kernel module. This can be achieved by module blacklisting from the kernel. This is useful in certain situations, such as when using a certain module poses a security risk to your system. One has to modify the `/etc/modprobe.d/blacklist.conf` to blacklist modules. The file `/etc/modprobe.d/blacklist.conf` is only editable by the root user.

11.4 Idea and Implementation of the Solution

Thus we can secure our system against the attack described in this section, in 2 ways.

1. Blacklist the kernel drivers which are used by the mount command to mount device files

2. Disable write and read access to the `/media` and `/mnt` directories for the user, after which the mount command fails to work as the user doesn't have required permissions to write to the `/media` directory or `/mnt` directory

Thus implementing the above two measures helps us secure the system against the attack described in the section by making the user require root access to mount an external media device.

12 Disabling Network Access

12.1 Attack Scenario

Now consider the same scenario with two users M1 and M2, M1 and M2 cannot share their keys with each other and thus M1 cannot get access to M2's submissions or M2's git repository even if they share keys with each other.

But without network restrictions in place, all our current security measures are naught. As M2 can upload his/her files to any file sharing service via the internet, and M1 can download them via a link to M2's uploaded files. Thus giving M1 access to M2's submission.

So, meaningful network restrictions have to be kept in place, not to allow any user to access the internet, except to our servers, so that he/she may not be able to share his/her submissions with anyone else.

12.2 Solution for the Attack

12.2.1 iptables

Iptables is an application that allows users to configure specific rules that will be enforced by the kernel's Netfilter framework. It acts as a packet filter and firewall that examines and directs traffic based on port, protocol, and other criteria. Iptables requires elevated privileges to operate, and they must be executed by user root. Otherwise, changes can't be made to rules present in iptables. Iptables thus allow the root user to define tables containing chains of rules for the treatment of network packets. Packets are processed by sequentially traversing the rules in chains. These rules contain specifications, with which if the packet matches, the specified action is executed

12.2.2 UFW

The Uncomplicated Firewall (ufw) program is designed as a frontend over iptables. Instead of writing rules for iptables which are hard to learn and complicated to analyze, ufw provides a simple and intuitive interface for configuring our iptables/firewall rules.

12.2.3 Main Idea

To stop users from accessing the internet, we have to use the ufw rules to drop all connections over ethernet. But since talking to our server requires internet access, we have to leave our server's IP address and corresponding ports required open. Since the rules for internet packet processing are applied at the kernel level, and the user can't override them without root access, our system is able to provide security in this regard.

12.2.4 Implementation

The following are a list of ports required by the coffre server. These ports have to be open at the client for ssh, git and mandos programs to work.

Ports Required			
Program	Port Required	IP address	Reason
OpenSSH	22	SERVER-IP address	SSH uses only port 22
Git	9418	SERVER-IP address	For the public Git repositories
Mandos	9601	SERVER-IP address	Required for Mandos to connect to the server

These restrictions ensure no access to internet connectivity other than with that of the server and thus help in stopping the attack discussed above.

13 Preventing Man In The Middle Attack

13.1 Attack Scenario

As in the above cases, let's take two users M1 and M2. All the above security restrictions have been put in place, i.e., a user cannot have root access, is not able to access the internet except connecting with our servers and cannot mount any external media.

13.1.1 IP spoof Attack

In the IP spoof Attack, an attacker who is between the client and server, and can control the network of the client, configures the DHCP server which gives IP address to machines in the network as that of the client, in such a way that the attacker is given the IP address of that of the server, while the server is removed from the network.

Since the attacker is able to control the client's network, removes the server from the network and poses himself/herself as the server and starts communicating with the client.

13.1.2 Man-in-the-middle Attack

In man-in-the-middle attacks (MITM), the attacker sits in the middle of the communication between client and server. The Attacker spoofs as the server to the client and as the client to the server. He can potentially read the network communication/files transferred between the client and the server and he can also modify them.

13.1.3 SSH defences

OpenSSH, on its own, detects these kinds of IP spoofing attacks and Man-in-the-middle attacks when public and private key authentication is used as the mode of user authentication, instead of password-based authentication. Every time the SSH program connects to the server, it checks the server's validity via a fingerprint based on the server's public key. Due to cryptographic guarantees, no one other than the server can produce a valid fingerprint.

13.1.4 Problems with SSH defences

The above mentioned two attacks can occur on our system even with all the security measures placed until now, as the above two attacks do not require anything else, but an IP address to which the Client connects to occur and our client should connect to our server to function properly.

OpenSSH trusts the user/client of the SSH connection and reports any inaccuracies/mismatches of the server's fingerprint to the user via a warning. In this way, it detects IP spoofing and man-in-the-middle attacks. The problem arises as the user can choose to ignore the warnings generated by the SSH program and continue with the authentication anyway. SSH also has options for the user to specify the server's public key for fingerprint verification, other than the default ones present in `/.ssh/known_hosts`. `known_hosts` file contains a list of server's IP addresses and associated public keys.

Whenever an SSH connection is made to a server, based on the server's IP address, and the IP addresses presence in the `known_hosts` file, SSH program checks against the relevant public key fingerprint and either issues a warning or accepts the connection silently.

13.1.5 Main Attack

Since, in our case, the user can't be trusted, and the OpenSSH program allows the user to specify an alternate `known_hosts` file and also a possible action when server's public key fingerprint does not match with what is present in the `known_hosts` file.

Consider the scenario with two users M1 and M2. M2 initiates a connection to the servers. M1 tries to perform an IP spoofing attack by posing himself/herself as the server. M2 gets a warning, or the M2's connection is closed by the OpenSSH program, warning M2 of a possible attack.

M2 specifies an alternate `known_hosts` file containing M1's public key fingerprint instead of the server's with the same IP address as that of the server. OpenSSH program validates the connection as

M1's public key fingerprint matches to that present in the new `known_hosts` file. Now an SSH connection is established between M1 and M2. M1 and M2 can share files happily between them.

13.2 Main Idea and Implementation of the Security Measures

Stop the OpenSSH program from taking in any additional parameters such as a `known_hosts` file, or any additional parameters. OpenSSH also provides a config file, an alternative way to specify options from a file instead of specifying them via a command line. Place the required options in the file such as `StrictHostKeyChecking yes`, which closes the SSH connection when an IP spoof or man-in-the-middle attack is detected instead of specifying a warning to the user and continuing on with the SSH connection.

For disabling the command line options to specify an alternative `known_hosts` file or an alternative config file or options for disabling the options mentioned in the config file, we can go into the source code of the OpenSSH program, and edit the code which processes these options. The code is present in `ssh.c`. It is modified in such a way that, even though the user specifies command line options they are ignored, and the location of the `known_hosts` and config file should be kept encoded in the binary itself.

Since the user is given only execute permission on the binary, and he/she requires a root level access to modify the binary, he/she cannot change the above-hardcoded values in the binary. The user is also given only read permission on `known_hosts` and config file, but not write access, so that he/she cannot change these files even if they want to, without root access.

14 Enabling Grub Security

Recall that all our security enforcements and guarantees mentioned in previous sections were based on the assumption that the user would not be able to get root access.

14.1 Attack Scenario

GRUB: GNU GRUB (short for GNU GRand Unified Bootloader, commonly referred to as GRUB) is a boot loader package from the GNU Project. Ubuntu 16.04 used as the Guest OS in our project uses GRUB as it's bootloader

GRUB has various options to let normal users get root account privileges, this is to be used when/if the administrator forgets his root password and wants to recover root account privileges

Two such methods for any user to gain root access are given below.

14.1.1 Booting into recovery mode

1. Switch on the computer.
2. Wait until the UEFI/BIOS has finished loading, or has almost finished. (During this time you will probably see a logo of your computer manufacturer.) Note: UEFI fast boot may be too fast to give time to press any key.
3. With BIOS, quickly press and hold the Shift key, which will bring up the GNU GRUB menu. (If you see the Ubuntu logo, you've missed the point where you can enter the GRUB menu.) With UEFI press (perhaps several times) the Escape key to get grub menu.
4. Select the line which starts with "Advanced options".
5. Select the line ending with "(recovery mode)", probably the second line, something like:
6. Ubuntu GNU/Linux, with Linux 3.8.0-26-generic (recovery mode)
7. Press Return and your machine will begin the boot process.
8. After a few moments, your workstation should display a menu with several options. One of the options (you may need to scroll down to the bottom of the list) will be "Drop to root shell prompt". Press Return with this option highlighted.
9. The root partition is mounted read-only. To mount it read/write, enter the command `mount -o remount,rw /`

10. If you have `/home`, `/boot`, `/tmp`, or any other mount point on a separate partition, you can mount them with the command `mount --all`
11. Use the `passwd` command to create a new root password.

14.1.2 Using GRUB to invoke bash

1. Select the appropriate boot entry in the GRUB menu and press `e` to edit the line.
2. Select the kernel line and press `e` again to edit it.
3. Append `init=/bin/bash` at the end of the line.
4. Press `Ctrl-X` to boot (this change is only temporary and will not be saved to your menu list). After booting, you will be at the bash prompt.
5. Your root file system is mounted as read-only now, so remount it as read/write `mount -n -o remount,rw /`.
6. Use the `passwd` command to create a new root password.

Both the above methods help any user with physical access to the machine, i.e., access to GRUB during booting gain root access on the machine.

14.2 Idea and Execution of the Solution

Since a normal user gaining root access defeats our security guarantees, we have to make sure grub doesn't allow any user to gain root access. This can be done by password protecting grub.

1. [Method to disable Grub Recovery options.](#)
2. [Method to disable editing kernel parameters by password protecting GRUB.](#)

By ensuring all the kernel boot options are password protected, and grub doesn't generate any recovery mode options, we can safely prevent this kind of privilege leak where the user gains root access via grub.

15 Full Disk Encryption

15.1 Mounting a filesystem on Live CD

A filesystem is a way that an operating system organizes files on a disk. These filesystems come in many different flavors depending on your specific needs. For Windows, we have the NTFS, FAT, FAT16, or FAT32 filesystems. For Linux, we have ext2, ext3, ext4 filesystems.

The Virtual Disk Image (VDI file) which is a Virtual Unix filesystem (ext4 journal) can be mounted on a live CD/pendrive, and data present on the virtual image can be accessed after acquiring root access on live CD/pendrive. Now as the user has root access on this filesystem, they can modify and add files into the VDI file. This attack is called Live CD mounting attack. In order to restrict this attack, we found out there is only one best solution, that is full disk encryption of the virtual image.

15.2 Full Disk Encryption (FDE)

While installing Ubuntu from the ISO file, Ubuntu asks to Encrypt the new Ubuntu Installation for Security. When the Encryption (Full Disk Encryption) options are selected, you will have to specify a security key or passphrase that will be used to encrypt and decrypt the disk. By doing so, every time the virtual image boots on a hypervisor, it asks to enter the encryption password.

15.3 Usability problem with FDE

As we are using the same basic VDI file which is encrypted for all the users/groups, the passphrase for decrypting the disk is the same for all the groups. As we can't give the encryption passphrase to the users (otherwise they can again mount using the same passphrase), someone has to go and type the passphrase every time the virtual machine is boot in. This is impractical.

15.4 Solution

As we can't give the encryption passphrase to the users/students, one of the solutions is to get the encryption passphrase from the network while the virtual machine is booting in and to automate the booting of the machine with the encrypted virtual image. There is an application called Mandos which does exactly the same.

16 Mandos

Mandos is a set of tools for allowing servers with encrypted root file systems to reboot unattended by getting passphrase from the network. It connects to a local or remote server to authenticate the user and sends the encryption password only after it authenticates the user machine successfully.

The Mandos system allows computers to have encrypted root file systems and at the same time be capable of remote and/or unattended reboots. The computers run a small client program in the initial RAM disk environment which will communicate with a server over a network. All network communication is encrypted using TLS. The clients are identified by the server using a TLS public key; each client has one unique to it. The server sends the clients an encrypted password. The encrypted password is decrypted by the clients using a separate OpenPGP key, and the password is then used to unlock the root file system, whereupon the computers can continue booting normally.

16.1 How much extent Mandos fits for us

We can have Mandos client running on the encrypted virtual machine of the user (instead of the server) to connect to the network and to get the passphrase during boot time. But Mandos guarantees security only when the Mandos client is not powered off for a long time

16.2 How Mandos doesn't fit for us

Though our machine has encrypted root file systems, it's not a server. As mandos is designed for securely automating the booting of servers with encrypted filesystems, it is not fully compatible to boot our virtual machine. Mandos constantly pings its client to check whether the system is up or not and disables the client if it's off for a considerable amount of time. This works in the case of servers because they are expected to be up all time. But our virtual machines can be powered off for any amount of time by the users and we can't disable the client if they are powered off for some time.

16.3 Possible Attack

The mandos client and its keys are present in the initramfs file. It should not be inside the encrypted disk as it is the one that has to connect to mandos server and gets the passphrase. So initramfs is not encrypted.

As we can't disable the mandos client, if it is powered off for a long time in our case, the user may then mount VDI file on a live CD, gets the unencrypted initramfs, uncompress it and can get the mandos secret keys. Once the keys are out, he/she can use them to connect to the mandos server and gets the passphrase. Once the passphrase is leaked, the user can mount the encrypted disk also and can add/modify files in it. The system is broken. We don't have any solution for this problem as of now. Securing mandos and preventing the leakage of the encryption passphrase are yet to be done.

Part III
Server Tools & Security

17 Securing SSH connection on server

17.1 AUTHORIZED_KEYS file in ssh

The 'authorized_keys' file in SSH specifies the SSH keys that can be used for logging into the user account for which the file is configured. It is a highly important configuration file, as it configures permanent access using SSH keys.

An authorized key in 'authorized_keys' file is a public key used for granting login access to users. The authentication mechanism is called public key authentication.

Authorized keys are configured separately for each user - usually in the .ssh/authorized_keys file in the user's home directory on the server. However, the location of the keys can be configured in SSH server configuration files.

Technically, an authorized key looks like this:

```
command="/.ssh/script.sh group01",
no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDOtlxZdHG2wevXcbOv3T15mRw5HpIly66F+
qhqryW/nrC1wb4U0qT3AH9O/zNN7L422YjXTGiCiCsVO0dZYa4D+uq0fYgKyR+i4NcG+5WXsvRyo
vQKEqwwLYIZotyKhWzHdrlxZF6VY82pv146HMhNvBVqoaBfVjBrMKIu05Lj2qOPxiNhGiP927xuo
Eau9GQBLDHpa6SUMv5PpAt2vBwlcX+BoXpv2fJvv3BFTRbOkFFAxjnxWo8lNrEAKZYtDsQ3jZA3jG
XXdUSH9lX53154tHMUgpf2j9gxbgmTxigKVCdYAX0oaC7/vs10sygP1813
qE6R8R2SbuBaUrk7ON9 group01@virtualBox
```

Listing 1: Authorized Key

Above authorized key format is:

command="⟨script path⟩ ⟨args...⟩",⟨options⟩,⟨user public key⟩.

17.2 Disabling shell access, -X option to the user in SSH connection

When the client(user) successfully authenticates itself by public key authentication, the client may request things like allocating a pseudo-tty, forwarding X11 connections, forwarding TCP connections, or forwarding the authentication agent connection over the secure channel.

17.2.1 Port forwarding

Tunneling or port forwarding in SSH can be used to make TCP/IP connection to applications running on other ports on the server. SSH first creates a tunnel, a secure connection between an SSH client and server. Then it forwards the connection to TCP/IP applications (client and server) communicate over the tunnel. But we don't want this, as users can try to communicate with other ports on the server. So we disable it using 'no-port-forwarding' option in the authorized key as shown above

17.2.2 X11 forwarding

The X11 forwarding feature provides one way for an SSH connection to access graphical applications running on the SSH server. X11 forwarding refers to executing a program remotely through an SSH (Secure Shell) connection. Meaning, the executable file itself is hosted on a server machine rather than where the graphical interface is being displayed (user machine). The graphical windows are forwarded to the user machine through the SSH connection. But we don't want to give the user any graphical access as of now. So we disable it using 'no-X11-forwarding' option in the authorized key as shown above.

17.2.3 Agent forwarding

When the user has permission for agent forwarding, he/she can forward the SSH connection to another host (let's say H)(not server),and the connection can be made from that host(H) to the destination (server). Any user(in our case its the user itself) on that host(H) with the ability to modify files could potentially use this key to authenticate and transfer files to the server (make submissions). As we don't want this to happen, we prevent agent forwarding using 'no-agent-forwarding' in the authorized key as shown above.

17.2.4 Pseudo-tty access

If the user has access to a pseudo terminal, he/she gets a terminal to type his/her shell commands which will be run on the server. By giving this permission we don't have control over the user's commands. So we prevent the user from getting a pseudo-shell using 'no-pty' option in the authorized key as shown above.

```
command=" /ssh/script.sh group01"
```

We don't want users to run any arbitrary command on the server once they get authenticated themselves using their keys. We can monitor the command passed to the ssh connection through a script. The script can allow only valid commands to execute and rejects any bad commands or unexpected commands. This can be achieved using the 'command=""' feature in the authorized_keys file. Inside the quotes, we have the path of the script to be executed, followed by any fixed arguments we want to pass to the script based on the user's identity (Eg. groupno in our case). We can access the command passed to the ssh connection in the script using a special variable called \$SSH_ORIGINAL_COMMAND.

The command is followed by the options, which are followed by the ssh public key of the user followed by the comment (group01@virtualBox in the above key) which is optional.

18 Git Server

Git server is a machine used to store files while working with Git. A Git server can be setup just by installing Git and then your users could connect to it as their remote. Remote is a place where the remote copies of the repositories are stored

We hosted a local git server on our server. All the connections (pullpushclone) requests are handled by the user 'git' on the server side just like Github. We can observe while one cloning a repository from the GitHub they write git@github.com:repository-path. Here git is the username and github.com is the server address.

We will be using usergroup interchangeably in the below sections because as of now our user is a client Virtual machine which is owned by a lab group/team.

18.1 Why local git server

The repositories must be access controlled, i.e. a user M1 shouldn't be able to access M2's remote git repository. The git server maintains private repositories of each group where they can sync their code every time they worked on their assignments. Only the group corresponding to a repository can read/write into the repository. No other group can have read/write access to the other group's repositories. This level of customized access control can be achieved, only if we hosted a git server. The access control is achieved by a tool called **Gitosis**

18.2 What is Gitosis

Gitosis is a tool which provides access control and remote management for hosted Git repositories. It allows for fine-grained management of read and write access over SSH, without requiring that the users have local system accounts on the server. To do this, it sets up a single system account "git" which is then used for all Git access.

Gitosis is basically a set of scripts that help us manage the authorized_keys file for the user 'git' as well as implement some simple access controls. The UI for adding people and determining access isn't a web interface but a special Git repository called 'gitosis-admin'. Only admin has access to that 'gitosis-admin' repository. We set up the information in that repository; and when you push it, Gitosis reconfigures the server and access controls.

This special repository 'gitosis-admin' contains a directory 'keydir' and a file 'gitosis.conf'. The gitosis.conf file is the control file where we specify users, repositories, and permissions. The keydir directory is where we store the public keys of all the users who have any sort of access to the repositories - in our case one repository per group.

'keydir' directory contains the public keys of all the groups each stored in a file groupno.pub. 'gitosis.conf' file contains the access control rules like which user has access to which repositories.

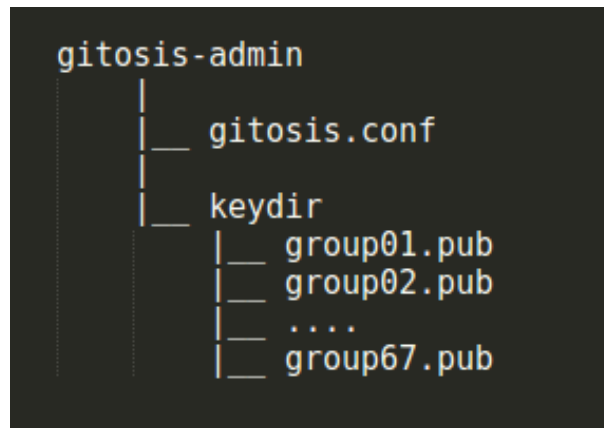


Figure 2: Gitosis-admin file hierarchy

```
1 [gitosis]
2
3 [group gitosis-admin]
4 members = admin
5 writable = gitosis-admin
6
7 [group grp01]
8 members = group01
9 writable = 01.git
10
11 [group grp25]
12 members = group25
13 writable = 25.git
14
15 [group TAs]
16 members = TA1 TA2 TA3
17 writable = A1-resources A2-resources A3-resources
```

Figure 3: Example of a gitosis-conf

18.3 What is Git Daemon

Git Daemon is a daemon serving repositories using the Git protocol. This is a common choice for fast, unauthenticated access to your Git data. Since this is not an authenticated service, anything you serve over this protocol is public within its network.

If you're running this on a server outside your firewall, it should be used only for projects that are publicly visible to the world. Git Daemon can be used in the case when projects/repositories have a large number of people or computers who have read-only access to them and when we don't want to have to add an SSH key for each computer.

Git Daemon normally listens on the port "DEFAULT_GIT_PORT" aka 9418. It waits for a connection asking for a service and will serve that service if it is enabled.

It verifies that the directory of the repository has the magic file "git-daemon-export-ok", and it will refuse to export any git directory that hasn't explicitly been marked for export this way (unless the `-export-all` parameter is specified). If you pass some directory paths as git daemon arguments, you can further restrict the offers to a whitelist comprising of those.

18.4 Handling public repositories

The git server also contains some public repositories which are the resource files required for every lab/assignments created by prof/TAs which can be only read by the students (no write access). We don't want to authenticate users to read these repositories which is an unnecessary burden on gitosis access control tools. So we use Git Daemon to host these read-only repositories to the public.

19 Various Client applications for handling submissions

19.1 Professor client application

A Web Client is provided to the professor to add new labs for the course and modify the existing labs. The Web client is a customized Django admin website through which the professor can add data corresponding to the labs which will be stored to SQL tables by the Django backend. The attributes of each lab comprise of the lab name, submission details, deadline of the lab, whether late submission allowed for the lab or not. The data stored in the SQL tables will be later used by the submission server when the user/student tries to contact it to make the submissions

19.2 Moodle Student client application

A client-side terminal application called (moodle) is provided to the users in their Virtual Machine for making the submissions easy. The terminal application has with similar control flow as of the Moodle website. It communicates with the server script for doing various actions. The server script exits the ssh connection immediately as if it receives an unknown/unexpected command. Using the client-side application, the students can see the list of available labs, the status of the lab (submission details, deadline, late submission allowed or not etc), make a submission and download the submission.

The application provides abstract commands for seeing the available labs, logistics of a lab and to make submissions and download submissions. It uses SSH and SCP in its code to authenticate the user and to run the appropriate commands.

19.3 Moodle TA client application

A similar terminal application is made for the TAs for seeing the available labs, to view submissions of the students for a lab, the logistics of the lab and to download the submissions.

20 Submission server

All the submissions are handled by the user 'submissions' on the server. The submissions occur through an SSH connection to the server. The SSH connection made by a user can be controlled using a file called `authorized_keys` located at `$HOME/.ssh/authorized_keys` on server side (`$HOME` is the home directory of the user 'submissions'). An entry in an `authorized_key` file looks like this:

Technically, an authorized key looks like this:

```
command="/.ssh/script.sh group01",
no-port-forwarding,no-X11-forwarding, no-agent-forwarding,no-pty
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDOtlxZdHG2wevXcbOv3T15mRw5HpIly66F+
qhqryW/nrC1wb4U0qT3AH9O/zNN7L422YjXTGiCiCsVO0dZYa4D+uq0fYgKyR+i4NcG+5WXsvRyo
vQKEqwwLYIZotyKhWzHdrlxZF6VY82pv146HMhNvBVqoaBfVjBrMKIu05Lj2qOPxiNhGiP927xuo
Eau9GQBLDHpa6SUmV5PpAt2vBwlcX+BoXpv2fJvv3BFTRbOkFFAxjnxWo8lNrEAKZYtDsq3jZA3jG
XXdUSH9IX53154tHMUgpf2j9gxbgmTxigKVCdYAX0oaC7/vs10sygP1813
qE6R8R2SbuBaUrk7ON9 group01@virtualBox
```

Listing 2: Authorized Key

Above authorized key format is:

`command="<script path> <args...>",<options>,<user public key>.`

The above entry enforces that the user corresponding to the above public key can't perform port forwarding (no-port-forwarding option), can't get shell access (no-pty option), can't access any GUI on the server using -X option (no-X11-forwarding option). Also, user has only had permission to execute 'script.sh' with some fixed arguments. The command given to the ssh connection is accessed in the script file through a special environment variable `SSH_ORIGINAL_COMMAND`. The script allows only valid commands to execute and marked as safe and rejects any bad commands or unexpected commands.

20.1 What is SCP

Secure copy protocol (SCP) is a network protocol which supports file transfers between hosts on a network. SCP uses Secure Shell (SSH) for data transfer and uses the same mechanisms for authentication,

thereby ensuring the authenticity and confidentiality of the data in transit. A client can send (upload) files to a server, optionally including their basic attributes (permissions, timestamps). Clients can also request files or directories from a server (download). SCP runs over TCP port 22 by default.

20.2 SCP for file submissions

We use SCP to transfer files from the user Virtual Machine to the server and vice-versa. As SCP uses SSH for authentication, we can authenticate the user while submitting files in the same way as he/she gets authenticated when he/she runs a command using SSH. As of now, we restrict the user to submit only tar.gz files for uniformity.

20.3 Changing timestamp attack and its prevention

The timestamp at which the user makes the submission for a lab is retrieved by the last modified time of the file the user had submitted on the server. SCP provides an option '-p' that preserves modification times, access times, and modes from the original file. The possible attack is that the user can intentionally try to make the submission with the SCP along with the '-p' option instead of using the moodle client and change his/her submission time.

This attack is prevented by checking the command the user had sent to the server in the server script before executing and rejecting it if the command is the 'scp' command and it contains '-p' option. In a similar way, the server also rejects '-r' option in SCP which is used for submitting a directory recursively.

20.4 Changing filename attack and its prevention

The user can give the destination filename, the name with which the file has to be saved on the remote server in an SCP command. The possible attack here is a group/user can try to make a submission with the filename as that of other group/user.

This attack is prevented by the following:
In the authorized key of every group/user, a fixed argument which is their 'groupno' is passed to the script and the destination path in the SCP command is changed to the groupno.tar.gz. So whatever name the group/user submitted file has, if it is submitted by a group G, it will be stored as G.tar.gz on the server.

20.5 Server overloading

If someone tries to upload gigabytes of data onto the server, it will become a huge load on the server. To prevent this, we kept a restriction of 50 MB on each submission for a lab per user and rejecting the submissions which have size more than that.

20.6 Handling partial submission

If a network problem occurs in the middle of the submission, we can't overwrite the old submission with this partial submission and say the last submission time is that of the partial file submission time. This problem is solved in a two-step way:

1. When moodle client tries to transfer the files to make submission for a lab using SCP, the server script checks things like whether the deadline for the lab submission is over, late submission allowed or not etc and terminates if submission is not allowed and stores the file in a temporary file with their groupno(eg: temp_group01.tar.gz) in the corresponding lab directory. This is like a draft submission(just like in IITB moodle). It doesn't affect actual submission.
2. If the first step fails, the moodle client throws an error. Else if the first step returns with exit code 0 (success), then the moodle client sends ssh command to save the draft submission. Then the server script moves the draft submission to the original submission.

20.7 Handling simultaneous submissions by the same group

If a group tries to make two simultaneous submissions, the first submission that is completed will be considered, and the other is rejected. The moodle client throws the error that 'your submission is rejected due to another parallel submission' or 'Error encountered while making submission'.

20.8 Handling simultaneous submissions by multiple groups

Theoretically, there is no limit on the number of parallel SSH connections. The only limit is the speed of the SSH connection as the network bandwidth is distributed between all parallel connections. So the parallel submissions are taken care of by SSH protocol.

20.9 Interaction of server script with SQL database

The server script interacts with a SQL database to get the list of labs, lab descriptions, deadlines, late submission allowed or not etc. That information is entered into the database by the professor/TAs using a website(Django web client) provided to them. The script interacts with the SQL server using SQL clients modules like pymysql.

20.10 SQL injection attack and its prevention

The user can try to make a SQL injection attack, by passing a SQL code in the place of lab name while seeing the list of labs or the logistics of the labs etc. This is prevented by taking care that strings passed to a SQL command will remain as strings even if it contains some SQL code.

Part IV
Client Setup & Update

21 Setup Script

21.1 Why do we require the setup

setup is a one time process, every user/group has to run on the base virtual machine (which is provided to them) to configure it to be usable by their group. The setup takes the groupno, and password as arguments contacts the server using common keys present in the base virtual machine and authenticates the group using the password. If the authentication is successful, the server sends the private keys corresponding to the group to the virtual machine, and once they are successfully installed, the virtual machine is configured to be used by their group.

21.2 Initial approach

Initially, we thought of providing preconfigured VDI files (disk file for the virtual machine) to each group for making submissions. But as each VDI file is of size approximately 6.5 Giga Bytes and the number of groups for the course will be around 70, the total space needed for storing VDI files for all the groups will be around 450 GB. Also, the creation of all this preconfigured VDI files is a time taking process. This makes this approach impractical.

21.3 Optimal approach

Instead of storing all the preconfigured VDI files on the server, we can store only the base VDI file on the server which is around 6.5 GigaBytes size and ask the users to run the initial setup on the base VDI file to configure it to their group after due authentication.

21.4 Securing the setup

During the setup process, the 'setup' downloads the ssh keys and the ssh binary file which contains the passphrase of the private key corresponding to the user. Later the keys and the binary file will be installed in their corresponding directories with the required permissions. The user has only execute permission on the ssh binary file (no read or write permission) after its installation. But during the setup process, if the user gets read access to their own ssh keys and their binary file, the user can use them to connect to the server using those files and submit the files from outside the virtual machine.

This attack is prevented in the same way updates are secured. The 'setup' script first creates a 'secure box' directory with permissions as 700 with root as the owner (in /usr/local/bin). That is, the user doesn't have access to read, write, execute the directory. Now the ssh files are downloaded into the 'secure_box' directory. The files are installed on the system (VM) only after the permissions of all the files are secured inside the 'secure_box' directory.

21.5 Another possible attack

The user may write code in the base virtual machine (without being configured for any group), and can later share a copy of the disk file(VDI file) to his/her friend. Later both the user and his/her friend can configure this virtual machine which has the user-written code to their corresponding groups. Now both the groups have the same code, and they are successful in copying their code.

This is prevented by removing write permissions for the user to all the files and directories on the base VDI file (even the tmp directory). The user can't create or modify any files on the base virtual machine without doing the setup process. Also, lightdm (login manager and the Ubuntu desktop) is disabled in the base virtual machine, and the user has only access to a tty shell makes it difficult to write any reasonable amount of code. The lightdm and the write permissions to the home directory of the user and the /tmp directory are enabled at the end of the 'setup' script.

21.6 What if 'setup' script is abruptly stopped in the middle

The order of tasks the 'setup' script does is examined and placed carefully in such a way that security is not broken even if the setup script is stopped abruptly. For example, giving write permissions to the home directory and enabling lightdm manager are done only after keys corresponding to the group/user and installed successfully.

22 Updates

22.1 Why updates

What if we find a security hole in the virtual machine provided to the user? What if we want to update the Moodle client application on the virtual machine?

We cannot tell the user that his old VDI file (virtual disk image) is no longer invalid and ask him/her to setup a new VDI file to be used by the group from then onwards. So we need an application on the user virtual machine which will check for the updates and install and execute the updates securely on the system to keep the security standards and the Moodle client like applications up-to-date.

22.2 Sudoers file

The sudoers file is a file Linux, and Unix administrators use to allocate system rights to system users. This allows the administrator to control who does what. Linux is built with security in mind. When you want to run a command that requires root rights, Linux checks your username against the sudoers file. This happens when you type the command `sudo`. If it determines, that your username is not on the list, you cannot run the command/program logged in as that user.

What you will have to do is log in as root by using the command `su -l`. The `-l` means it should log in normally. The default user for the `su` command is root. Then you will enter the password for the root account, giving you a shell prompt where you can run any command as root. Again, this not safe. Once you are logged in as root, the system is open to vulnerabilities. It is best to supply rights to the non-root user for the sole purpose to run a desired command/program.

22.3 How we are using sudoers file to run update script as a root user

The user using our virtual machine is a non-sudo user.

In sudoers file, we can specify that a non-sudo user can have permission to run only a particular script located at a particular location as a root user.

The update script is located in `usrlocalbin` directory, and the user neither has write permission to that directory nor to the update script. So he/she neither can modify the update script nor can delete or rename the update script. Now we can give the user permission to run this update script as a sudo user.

22.4 Version Control

When the user runs the update script on the Virtual Machine, it contacts the server, downloads the latest compressed updates file. The latest updates contain the commands file to run on the virtual machine and the supportive files to install on the machine. The commands file contains code to make security fixes and the code to match the existing version of Moodle client application on the machine with the latest version and install the latest version if an older version is found or the application is not found on the machine.

22.5 Securing the updates

When the 'update' script downloaded the compressed updates file, if the user got write access to that file somehow, he/she may modify it, to run the commands of his/her interest with the root permission as if they are part of the update. In order to prevent this, the update script first creates a 'secure box' directory with permissions as 700 with root as the owner (in `usrlocalbin`). That is, the user doesn't have access to read, write, execute the directory. Now the updates are downloaded into the 'secure_box' directory. The files are installed on the system (VM) only after the permissions of all the files are secured inside the 'secure_box' directory.

Part V
Conclusion

23 Preventive measures if Mandos passphrase is leaked

The following preventive measures are to be taken in case if the Mandos passphrase is leaked, and the user is able to decrypt the hard disk containing the system. Since the user can decrypt the hard disk, it is assumed he/she can access, read and modify all files in the system.

23.1 User identifies private key passphrase from SSH binary

Even if the passphrase is leaked and the user is able to decrypt the hard disk and thus can change any files easily, the user still has to use the SSH keys to authenticate himself with the SSH server.

Since the user can change any file easily, he can gain sudo access by changing the sudoers file. But since we have encrypted our private key and public key, the user has to still use our SSH program, to decrypt those keys, unless he runs the program on a debugger, follows the assemble code in the binary line by line and identifies the passphrase that is hardcoded into the SSH binary, which is required to decode the private key and use the private key for authentication.

In this case, the user is able to break into our system and can use the private and public keys present in the system on his desktop. We cannot stop such a kind of user, who has advanced knowledge and thus can easily spoof any checks performed by the binary for the server's sake. Unless any new technologies come up or SGX becomes usable, nothing can be done against such kind of users. We can periodically update the keys in our system so that the user has to undergo the same process again and again for retrieving the keys.

23.2 User cannot identify the private key passphrase from SSH binary

So, assuming the user cannot recover the passphrase from SSH binary, only our SSH binary can be used to authenticate the public and private keys. So, we can place checks inside the SSH binary.

23.2.1 Check if the SSH binary is running in a VirtualBox environment

Since, the user is using our SSH binary we can add code inside the binary, to check whether the SSH binary is running inside a VirtualBox environment or not. This can be done by looking at the processor info, loaded kernel modules, and available drivers. This helps us in detecting if a passphrase breach has taken place. We can report this breach immediately to the server which blacklists the user and notifies the instructor.

23.2.2 Check if the User has gained ROOT access

Since the user is using our SSH binary, we can check the contents of sudoer's file, which has to be edited by the user to gain sudo access and other critical configuration files in Linux which are present in /etc directory. Getting any additional accesses by the user such as gaining full internet access, results in the network configuration files in /etc being changed. Checking these files and other files integrity via a hashing algorithm and also checking their last access times, help us detect a breach if the user is not careful enough. We can report this breach immediately to the server which blacklists the user and notifies the instructor.

23.2.3 Check if the User has introduced any new files into the system

inotify is a program which can monitor all the filesystem events. It can tell when a new file or directory is created, accessed, read or modified. If the user decides not to get root access, but instead just decides to send additional files into the hard disk, without changing any file or directory permissions and without gaining any root access or firewall permissions, we can use inotify to detect the breach. We can use inotify to keep track of all files created by the user in the system when the system is powered on, and when a new file appears in the system, which has not been created in the system according to inotify, we can detect a breach has taken place. As soon as a breach is detected, info can be sent to the instructor about the system and the user causing the breach.

We hope that the threat of serious action by the instructor on the user if the user bypasses the system, will stop the user from trying to bypass the system, as one breach detection is enough to penalize him. It should be noted that the user can be able to bypass the checks performed by the SSH program if he knows exactly what checks are happening. Here, we hope the secrecy of the algorithm, about which checks are being performed helps us stop this scenario.

24 Software Guard Extensions (SGX)

24.1 Introduction to SGX

SGX is also known as Software Guard Extensions (SGX), developed by Intel for Intel Processors, are a set of security-related instruction codes. SGX allows user-level code to create private memory regions, called enclaves. These enclaves are isolated from other processes running at the same or higher privilege levels. The code running inside an enclave is effectively isolated from other applications, the operating system, the hypervisor, et cetera. Each enclave has its own protected memory, which can't be accessed even by any root level process on the system where the enclave is present. Thus enclaves can be used to store secure data on a system, where even people having physical access to the system cannot access the data.

The main use of SGX is to run computations securely across the internet, on a remote machine, with the guarantee that the data is not tampered with.

24.2 Current State of SGX

SGX is in a very nascent stage. Currently, the code running in an enclave can only be programmed in C/C++. Current SGX enclaves do not support all the system calls in the Linux kernel. SGX is only enabled for Intel processors from 2016, that too to only specific models of processors. This means SGX programs are not runnable on all systems in use currently, and the current gen systems may have to go through some life-cycles before SGX instructions can be used on all systems present.

It should also be noted that SGX is an invention by Intel, and so is only available on Intel processors.

24.3 How SGX can be used in our system

Currently, there are some directions by research groups, in using SGX, to create secure containers such as [Scone Containers](#) , in which programs are guaranteed to run securely, and untampered with by the user. Since our current limitations using conventional technologies have some flaw that can be used by the user, with physical access to the system, to exploit and gain root access, we hope that some work can be done in future using SGX and writing secure programs which cannot be tampered with even by a user with physical access to the system to patch out flaws in the current system.

But it should be noted that it may take a long time for SGX to be usable for our purposes.

25 Future work

25.1 Usability

Currently, this system is made to be used in a single lab for the lab groups to make submissions without being able to copy from one another. This system can be extended to support multiple labs without the user having maintaining separate VDI files for different labs. For being able to use the common VDI file for multiple labs, the group level authentication should be changed single student level. A middle layer can be used for managing student-group relationships

25.2 Student Montioring

This system can be extended to monitor students activity, such as taking their attendances and auto-grading their submissions. Since submissions are handled by our Moodle script, we can include validity checks on students submissions, forcing them to require a fixed submission format containing pre-specified files. We can also add an automatic plagiarism check for student submissions bu using MOSS, and send the results to the professor automatically.

25.3 Creating preventive measures if encryption passphrase is leaked

It is impossible to fully secure a system for which the malicious attacker has full physical access. More and more security layers can be built to cover these hole's which can make it extremely hard for the user to break into our system. Section 23 provides more ideas and methods using which security layers can be built. Keep in mind that the algorithms for the security layers have to be kept secret and

it is preferable for these security layers to be coded in C and have obfuscated binaries, so as to not leak the algorithms which are being used by them.

25.4 SGX

SGX is a new and exciting technology. It may have the potential to make our system fully secure even when complete physical access is given. Section 24 gives a brief introduction to SGX. Future work involves researching more about SGX and how it can be used to make our system fully secure.

26 Conclusion

It is impossible to fully secure a system for which the malicious attacker has full physical access. Since in our case, the users of the system have full untethered access to the Virtual Disk Image (VDI) file, which is equivalent to having full physical access on the system, we cannot truly secure it completely.

We cannot fully secure our system, but we can make it harder and harder for the end user to break our system by adding more and more layers of security checks one above the other.

In our project, we started with a normal Virtual Machine, with no security, started plugging security holes such as Network Access, USB Access. We have brought the system into a stage where almost all the security holes have been filled. But still, a security hole remains, as we can't stop the user from having physical access to our system. But still, more and more security layers can be built to cover this hole which can make it extremely hard for the user to break into our system.

Part VI

Coffre User Manual & FAQs

Coffre User Manual

Coffre is a BTP project started under the guidance of Prof. Uday Khedkar with the aim of providing the students an isolated, distributed environment where they can write their code and make submissions without copying from one another or from the internet. Coffre is a french word which means **VAULT**.

Each student/group gets their own copy of their virtual disk image (VDI) file with non-root access. They can load it on a Virtual Machine and start using it. The student can't interact with the host OS i.e can't copy text or files from host to VM or vice-versa , can't mount discs and can't connect to the internet except to a git repository provided to them and the submission portal to make submissions.

I. Setup

1. VirtualBox Installation

- Download and install Oracle VM VirtualBox from the [official](#) site.
- Refer to **Appendix** if you get any **errors** while installing or using the VirtualBox.

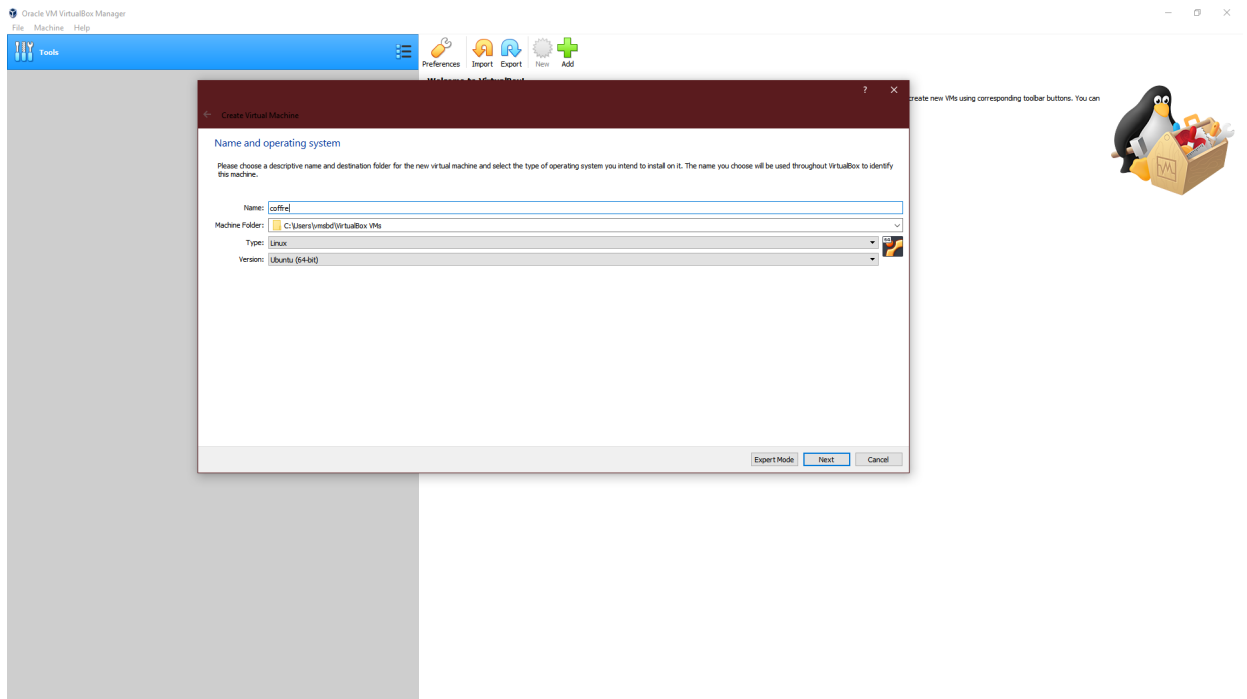
2. VDI download

- Download the VDI file from this [link](#) or from the link provided to you in the E-mail.
- Note your **group name** and **group password** which you have received through E-Mail.
- If you didn't receive the mail, kindly drop a mail to Prof. Uday Khedkhar with a cc to vamsi@cse.iitb.ac.in and bhavan@cse.iitb.ac.in

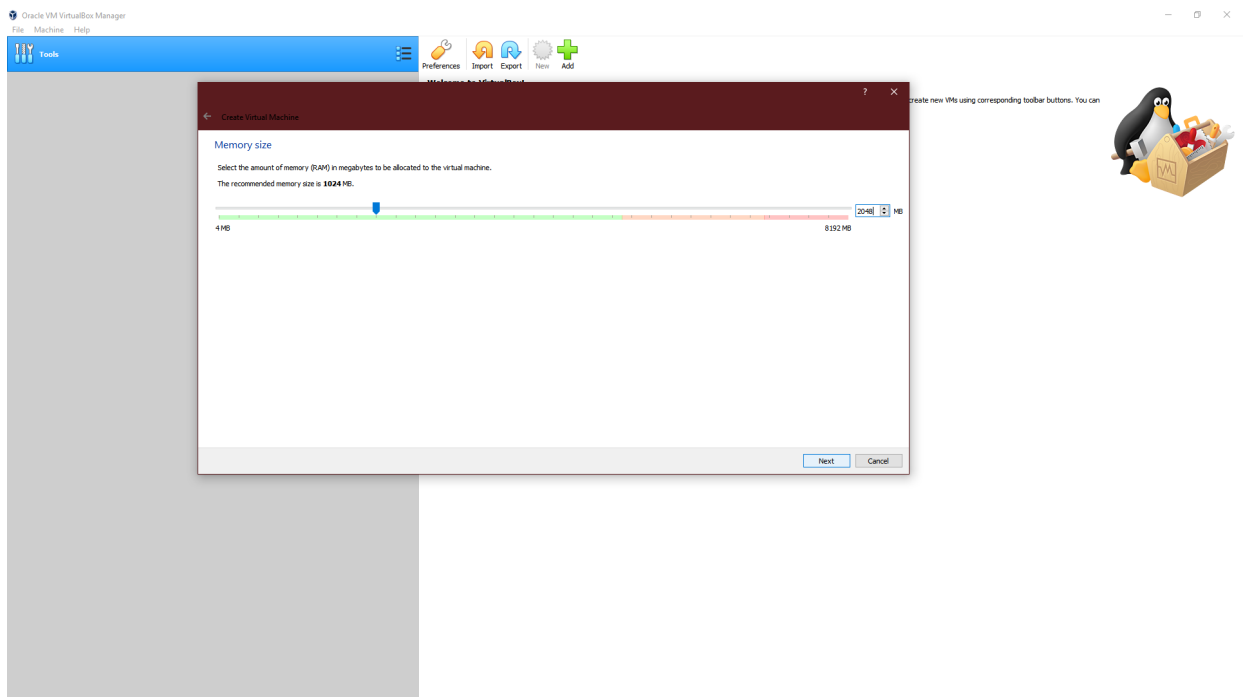
3. Virtual Machine setup

- Create a new virtual machine (VM) as follows:
- Click on the **new** button on the top of Oracle VM VirtualBox Manager
- Choose the **Name** of your choice for the VM , Operating System **Type** as Linux and **Version** as Ubuntu (64 bit) and click **Next**

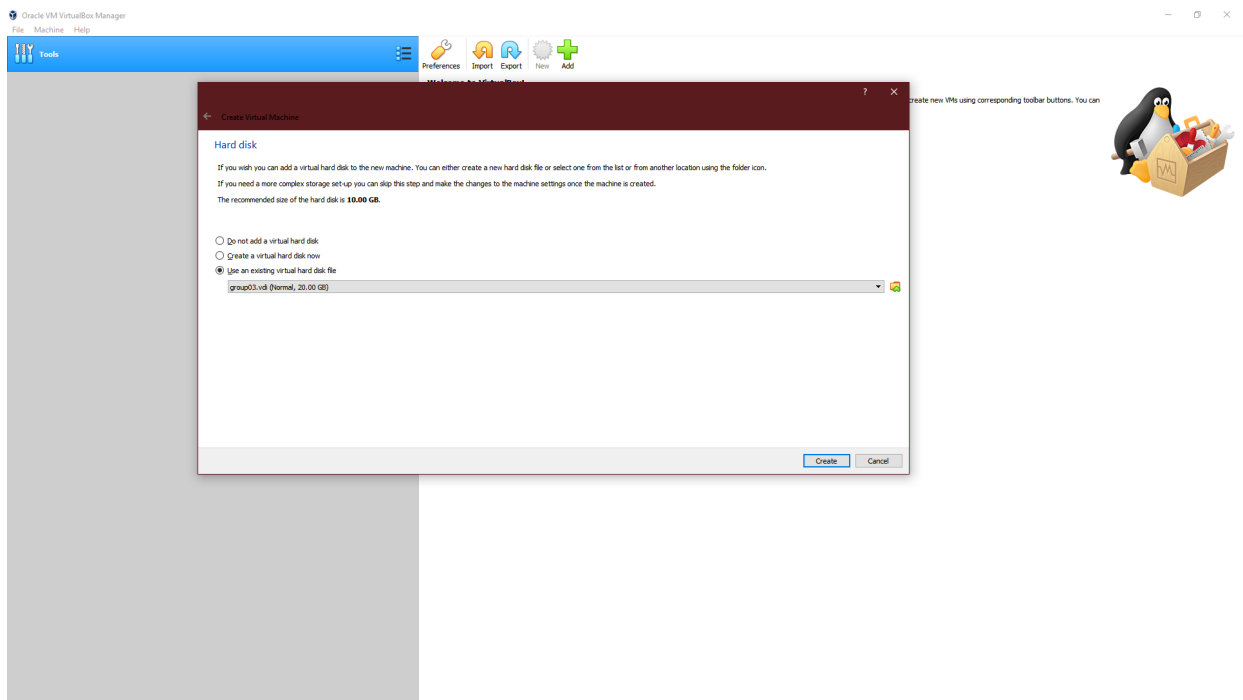
- Note: The OS provided to you in the VDI file is Ubuntu 16.04.



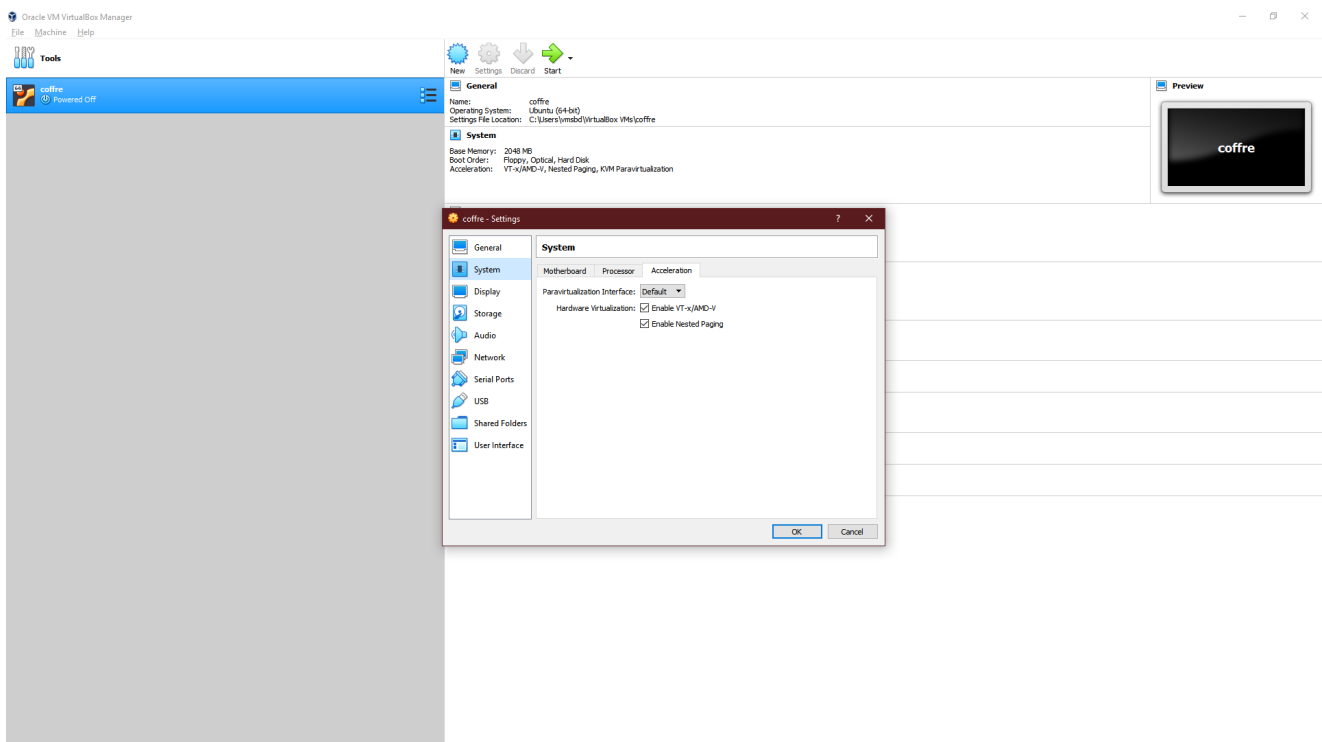
- Allocate atleast **2GB** RAM and click **Next**



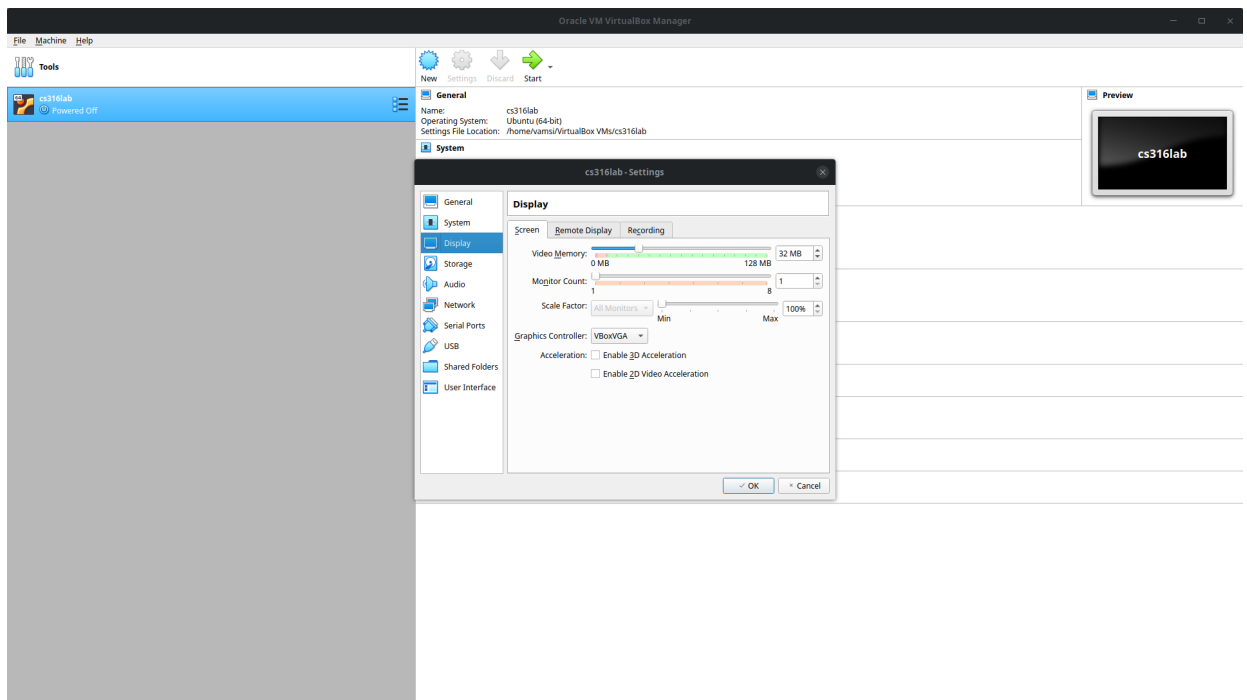
- Select **Use an existing virtual hard disk file**, choose the VDI file downloaded in the previous step and click **Create**.



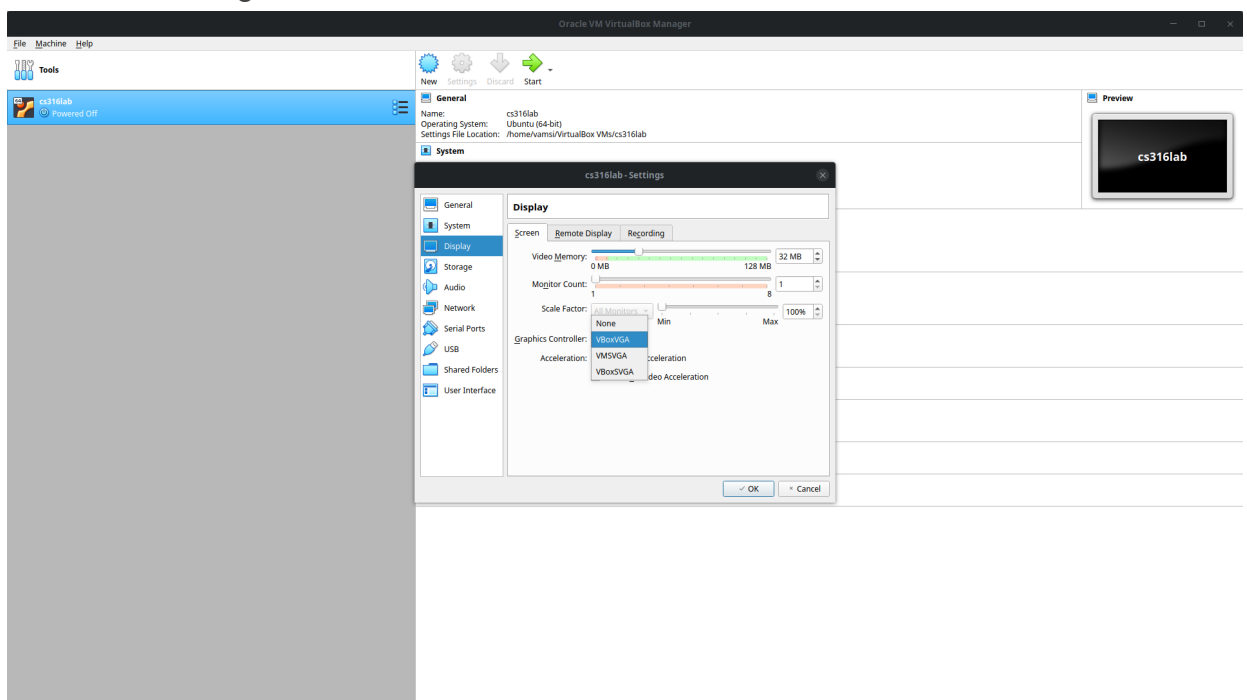
- After your machine has been created go to settings -> System -> Acceleration, make sure Enable VT-x/ AMD-V and Enable Nested Paging options are checked. This is to ensure that that OS running on VM has access to hardware acceleration.
- If you are unable to tick them (i.e if the options are freezed), then make sure **Virtual Technology** feature is enabled in the BIOS of your PC



- Go to Settings -> Display -> Screen and allocate it more than **32mb** virtual memory. This is to ensure that the VM can be run in Full Screen Mode



- If you are facing any issues due to screen resolution, go to Settings -> Display -> Graphic Controller and try different options. **VBoxVGA** is the recommended Graphic Controller Setting.



4. Virtual Machine Startup

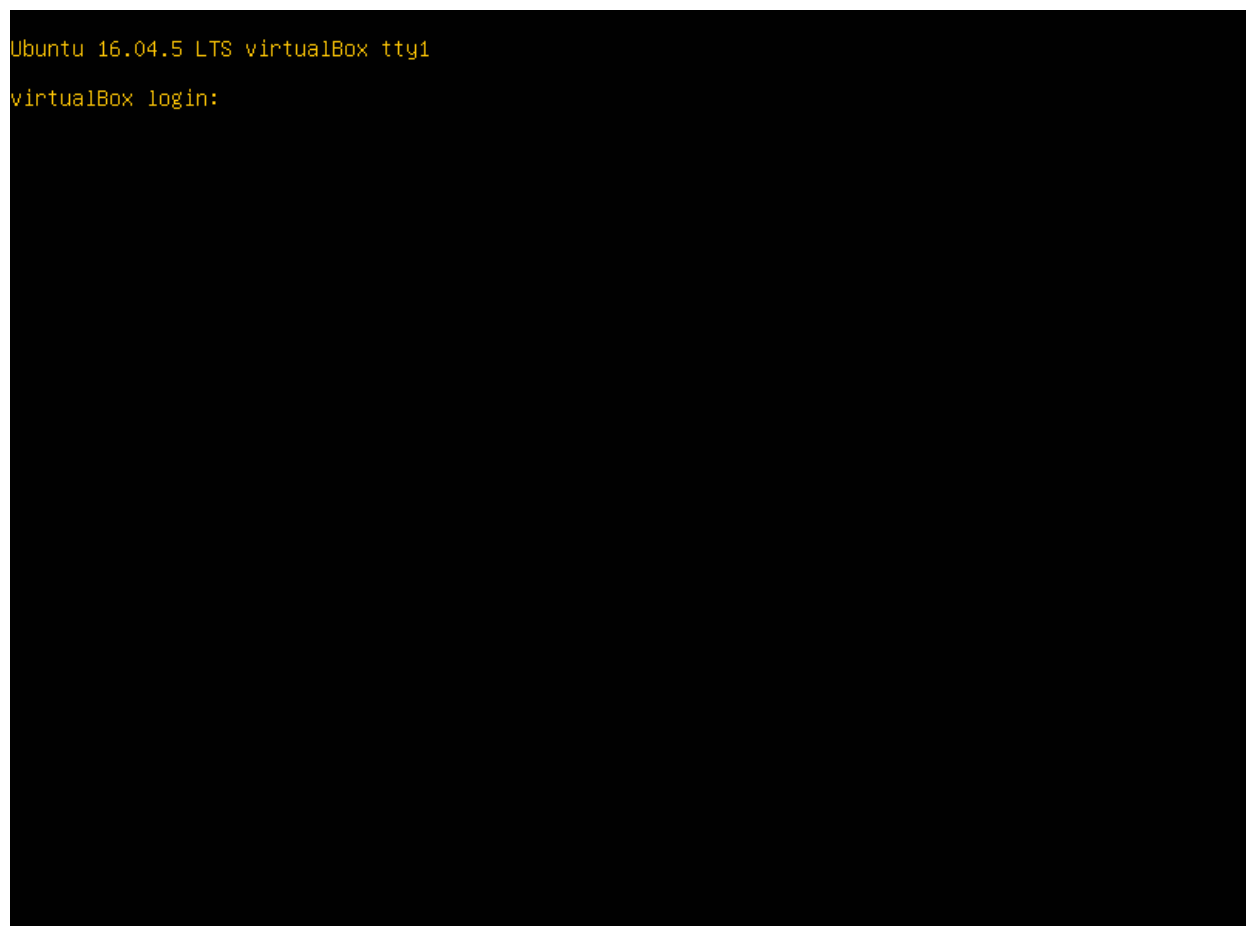
- Make sure you are connected to **IIT Bombay internal network**, before starting the machine
- Start the newly created Virtual Machine
- You will be presented with an enter passphrase screen, **do not press anything** and **wait for a minute**. The system would automatically obtain the passphrase from the

server and boots you into the machine.



5. Virtual Machine Initial Setup

- The Virtual Machine waits at the enter passphrase screen to obtain the passphrase from the server. **Do not press anything** and **wait for a minute**.
- After starting the Virtual Machine, you will be presented with a tty screen as shown below



- Login with the username: **basicuser** and password: **1234**
- After logged in, run `$ setup <group{number}> <group password>` in the terminal. The setup is a one time process to configure general VDI file to be used by you group.
- Wait for **10-15 seconds** for the setup to run. **Do not interrupt** the setup process in the middle. The system will reboot automatically after the setup run successfully.
- Now the system has been set and ready to be used by your group
- From now onwards, you can use your group credentials to login to the Virtual Machine
- After you first login, change the password of your login using the command `$ passwd` to keep your Virtual Machine secure
- Open a terminal and run `$ update` to get the moodle application which can be used for making the lab submissions.

II. Usage

1. Virtual Box usage

- Click **Right CTRL (Host key) + F** to go to full screen mode in the VM.

2. Commands supported inside the environment

All the default commands in the Linux terminal like `ls`, `mkdir`, `cat`, `nano` etc are supported. Apart from that, you can use **moodle** (a terminal application) for making submissions to the labs, **git** to connect to a remote git repository of you group to sync data between multiple copies of your group virtual machine instances, **\$ update** command to install the latest security updates and latest version of moodle application.

Note: As the server cannot be accessed outside IIT Bomabay as of now, **you have to be connected to IITB Network** to **use moodle, git, update command**.

3. Moodle usage

moodle is a command line application, through which you can submit your labs. Type `moodle --help` in the terminal for usage instructions.

- **Commands:**
 - To see the list of all available labs for the course `moodle --listlabs`

- To see the status of the lab (Deadline, late submission allowed or not, submission details etc.) `moodle --status --lab <labname>`
Example: `moodle --status --lab Assignment02`
- To make a submission to a lab `moodle --upload --lab <labname> --file <filename>`
Example: `moodle --upload --lab Assignment02 --file Assignment02.tar.gz`
Note: Only .tar.gz files can to be submitted. This is enforced to maintain uniformness in submissions. Any file you submit will be stored as `<group{no}>.tar.gz` on the server for that lab where `<group{no}>` is your groupno
- To download the submission for a lab
`moodle --download --lab <labname> [--file <filename>|<dirname>]`
Examples: It can be used in three ways:
 - 1) `moodle --download --lab Assignment02 --file Assignment02.tar.gz`
(Downloads your Assignment02 submission to Assignment02.tar.gz)
 - 2) `moodle --download --lab Assignment02 --file Desktop/` (Downloads your Assignment02 submission to Desktop/ as Assignment02_download.tar.gz)
 - 3) `moodle --download --lab Assignment02` (Downloads your Assignment02 submission to current directory as Assignment02_download.tar.gz)

4. Git usage

- **Git** can be used to sync your repository across your multiple virtual machine instances
- **Git usage:** For a group `<group{no}>`, **say group09**
 - To create your group repository for the first time:


```
mkdir group09
cd group09
git init
git remote add origin git@coffre:group09
touch readme
git add .
git commit -m "Initial commit"
git push origin master
```
 - To clone the repository: `git clone git@coffre:group09`
 - To push the repository: `git push origin master`
 - To pull the repository: `git pull origin master`

- All the git commands work as they are (eg. git log, git checkout, git status, git fetch will work)
- Reference implementations are provided via a public git repository . To download reference implementations use `git clone git://coffre/resources.git`
- Whenever, the reference implementations have been updated use `git pull origin mater` inside the resources directory, to pull the new reference implementations

III. Appendix

- If your VM is unable to start or giving out errors, see that hardware virtualization Technology features are enabled in your BIOS.
 - To check if hardware virtualization is supported by your cpu
run `sudo apt-get install cpu-checker`
then check `kvm-ok`
 - Ignore `/dev/kvm/ does not exist` error if it occurs
 - Check for KVM acceleration can be used or KVM (vmx) is disabled by your BIOS
 - If it is disabled reboot the system and turn it on in BIOS
- Refer to [this](#) page if you get '**modprobe vboxdrv' error** or **VERR_VM_DRIVER_NOT_INSTALLED (-1908)** error
- Refer to [this](#) page if you get **VirtualBox kernel modules do not match error**
- If you are still unable to use the system properly, send us a mail at vamsi@cse.iitb.ac.in or bhavan@cse.iitb.ac.in with relevant error information

Frequently Asked Questions?

Q) How to run a given VDI file ?

A) You, have to use Oracle VirtualBox program. In it create a VM and then attach the VDI file to the VM. You can then run the given VDI

Q) What OS is present in the VDI file?

A) Ubuntu 16.04, is present in the VDI file.

Q) How much space is required run coffre VDI?

A) Coffre VDI file itself is around 7GB in size. So, 10GB space is more than enough for using and running the Coffre VDI.

Q) Do I need internet access to use coffre?

A) Yes, During bootup of the Virtual Machine you need internet access. Specifically, you should be able to access IIT Bombay Internal Network.

Q) Can I use it outside IIT Bombay network.

A) No, not unless you have a VPN to access Internal IITB Network. The system needs access to this network during bootup.

Q) Can I copy and paste from Host to Guest (VM) using CTRL-C and CTRL-V?

A) No, copying from Host to Guest and vice-versa has been disabled.

Q) Can I access internet from inside the provided machine?

A) No, internet access is disabled inside the Coffre system.

Q) Can I insert USB stick into this machine?

A) Yes, you can but you cannot transfer any files from the USB stick into the Machine or vice-versa

Q) Can I make a copy of the VDI file configured by our group and use it?

A) Yes, you can do it. But make sure no UUID conflict occurs in VirtualBox.

Q) Can I get root access via grub?

A) No, this feature has been disabled.

Q) Why my system waits for a long time at 'Enter Passphrase' screen?

A) Your system has to contact a server and get the passphrase to login. So, it might take some time.

Q) How much RAM has to be allocated for ideal running of this machine in VirtualBox?

A) We recommend to allocate atleast 2GB of RAM to the machine

Q) What are the credentials to login into the tty terminal in base VDI?

A) Use the credentials username: basicuser and password: 1234, for intial login via tty.

Q) What arguments to be provided for setup command?

A) For setup command you have to provide your and associated .

Q) What credentials to be used to login after setup is done?

A) After setup is done, you have to use your and to login

Coffre - Professor manual

- Login with the professor username and password in the Django website
- Click on labs and add new assignments.
- Ask admin to add a new repo for the lab
- Add reference implementation in corresponding public git repositories for which only professor has write access which will be setup by the admin
(admin has to give professor write permission to the corresponding repositories in gitosis.conf file and add git-daemon-export-ok(empty file) inside the root directory of the repo to enable public read access as mentioned in the gitosis setup)

Coffre - File & Directory Structure

The following section explains the files & directory structure of the coffre server:

Django Server:

The code for the Django server is present in “/home/cs316local/django-server” folder.
The server can be started with “python3 [manage.py](#) runserver”.

Git Server:

The main configuration files for Git Server(called gitosis) are present in “/home/cs316local/gitosis-admin”

The directory structure is as follows:

- gitosis-admin
 - -----> gitosis.conf (Contains Our Git Server Users and their associated permissions for their repositories)
 - -----> keydir (Contains Public keys of the Git Server Users)
 - -----> group01.pub
 - -----> group02.pub

- -----> group03.pub

Submission Server:

The files for submission server are present in /home/submissions

- submissions
 - -----> Lab01 (Directory for group submission files for the corresponding lab)
 - -----> updates (Folder containing updates to be done)
 - -----> .ssh (Folder containing submission server's configuration files and required scripts)
 - -----> authorized_keys (Permission file determining which users are able to submit and give other access permissions to TA's)
 - -----> [script.py](#) (Script handling moodle application)
 - -----> [script.sh](#)
 - -----> [TAscript.py](#) (Script handling TAMoodle application)
 - -----> [TAscript.sh](#)
 - -----> setup_script.py (Script handling Initial Setup)
 - -----> setup_script.sh

User Data:

The User Data folder is present in /hom/cs316local/user_data

The files in this folder are used for password checking for groups.

This folder also contains Custom SSH binaries one binary for each user, which are put into their respective Virtual Machine's during setup

- user_data
 - -----> passwords_file (Contains usernames and passwords for all the groups present)
 - -----> all_keys
 - -----> basicuser
 - -----> group01
 - -----> id_dsa (User's encrypted Private Key)
 - -----> id_rsa.pub (User's Public Key)
 - -----> passphrase (Passphrase used to encrypt Private Key)

- -----> ssh_files (Directory containing custom ssh_binaries for each user)
 - -----> ssh (Main custom SSH binary)
 - -----> Other ssh programs
- -----> group02
- -----> group03
- -----> group04

Updates:

The files for updates are present in “/home/submissions/updates”

- updates
 - -----> [commands.sh](#) (Script file which is to be run on the client, when update command is used)
 - -----> <Helper_Files> (Helper Files which are required for [commands.sh](#) script, these may include .deb files for package installations)

Coffre - Admin Manual

Creating and adding users/groups to the server

The scripts for creating and adding users/groups to the server are present in /home/cs316local/keys-setup on server

The creation process is done in the following way:

- Remove all_keys, gitosis_files, authorized_keys_file, passwords_file before running scripts in scripts/
- Go to scripts/ directory
- Note: In every script at the top, there is a variable called ‘GROUPS_COUNT’ which is the number of groups you want to create keys for. Change it accordingly.
- Run the scripts in the following order:

1. `./key_generator.sh` (It creates ssh files and ssh keys(id_dsa, known_hosts, scp, ssh ,ssh-keyscan) for all users in a folder called all_keys)
2. `./create_gitosis_admin.sh` (It creates gitosis_files (keydir,gitosis.conf) (which controls permissions of repos to different users) which has to be pushed on the git server by admin as mentioned in the gitosis setup documentation)
Note: Add `cs316local@cs316local.pub`(which is the public key for `cs316local@cs316local.pub` who is the admin for gitosis) to keydir
3. `./create_authorized_keys_file.sh` (It creates authorized_keys_file
Paste/replace the authorized keys in `/home/submissions/.ssh/authorized_keys` for the groups after the authorized_key for `cs316local@cs316-ubuntu`)
4. `./passwords_file_generator.sh` (It creates passwords for all the groups in 'passwords_file')

Generated files 'all_keys' and 'passwords_file' are to be placed in `/home/cs316local/user_data` which will be used by the server to authenticate and send corresponding ssh files to the client when client runs setup.

Username and Passwords of the groups present in the 'passwords_file' have to be send to the corresponding members of each group through email so that they can use them to authenticate themselves while running setup and configure the base_VDI to be used by their group

How to add a new TA:

The TA keys have to be added on the server so that they can use moodle-admin to view and download the submissions to the labs by the students

The following authorized key has to be added/pasted at the end of authorized_keys file (`/home/submissions/.ssh/authorized_keys`)

`command="~/ssh/TAscript.sh",no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty`

Eg:

`command="~/ssh/TAscript.sh",no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty ssh-rsa`

`AAAAB3NzaC1yc2EAAAADAQABAAQBAQC+yoJSJ3G70qN6K9oITQqatuSuP`

QD2CiJV+E2VtUG+8qjzqoe6NyFvWo5aPx9UqtKT8iGk+ZncMCsmOOSHio+I1h
bSONqZyodLh7p0daYMPY3WvhZTiZorNiFS0f1iH0FpJ6eSugJXTwyoWIMzbmV
Q6TIRmVOtcqUXF2GKZI+/86rTQtV3FkQMkmZP1Fg2IlvLGpMqxOf5LVxEHd3
E00M1OHf3/7R+0aLi2rkoxMDCoozEkTGOBiEjslbrlQCcHRXwvUM9S97kq3ru
RrryD//HpY7hpJcTAJwOtQPfgyNwUW7cL/ZvSosU8eyOqRZG6QCGNxcYlu/h0
oES429abhN person@machine

Adding professor login and professor key to server:

Django login

URL is <http://10.129.6.46:8080/admin/>

Professor login credentials have to be added to the django admin website with access to the table 'Assignments' so that professor can add new labs/assignments or edit existing labs/assignments

Adding professor public key to gitosis-admin

Prof public key has to be added to gitosis-admin/keydir (eg. with the name 'prof.pub') and has to be pushed to the git server.

Adding a new lab repository

Professor has to be given access to a lab repository (say A1-resources to be used as a reference implementation files for Assignment A1) by the admin.

This can be done by giving professor write access to the A1-resources repository in gitosis.conf (Don't forget to push gitosis-admin after doing changes in gitosis.conf)

Eg.

```
[group professors]
```

```
members = prof
```

```
writable = A1-resources,A2-resources
```

where 'prof.pub' is professor's public key which is present in gitosis-admin/keydir and A1-resources (can have any name) is the lab repository for Assignment A1

Updates:

Admin can give updates to the VDI files of the clients (i.e groups)

'updates' folder is present in the home directory of submissions(/home/submissions) on server.

It has to be done in the following way:

- updates folder has to contain [commands.sh](#) along with any supportive files and folders.
- Put all the commands to run on the user's machine (commands run with sudo permission on user's machine when user run 'update' in terminal) in [commands.sh](#)
- Put any supportive files you need in updates directory.

When user run 'update', the updates directory will be transferred to user's machine and [commands.sh](#) will run using the supportive files with sudo (as a root user). You can change any thing on the user machine by writing the necessary script in [commands.sh](#)

Coffre - Source Code Structure

All the code and instructions to build Coffre, from the ground up using the code, are present in coffre_code directory

- coffre_code
 - base_vdi_client_setup_and_usage (Contains Scripts and README's for those scripts, to generate Coffre client)
 - README_CREATING_SeedVDI (See this README for instructions on how to create the SeedVDI)
 - README_BASEVDI_FROM_SEEDVDI (See this README for re-creating the BASEVDI from SeedVDI using the scripts)
 - README_SERVER_CHANGED (See this README for updating the client if the server has been moved)
 - server-setup
 - django-setup (Instructions for installing and setting up Django server)

- gitosis-and-ssh-server-setup (Instructions for installing and setting up git server and submission server)
- keys-setup (Instructions for generating and adding a new group/student)
- mandos-setup (Setup and Installation Instructions for Mandos Server)
- moodle_client_files (Program code for Moodle Client Files)
- server_basic_installations.sh (Scripts containing some basic installations for the server)
- README (Please refer to this README for server installation)